

**Neuro-Adaptive Network Flow Orchestrator: Deep Reinforcement Learning for
Proactive Congestion Control in Software-Defined Campus Networks**

(Internal)

Dhruvin Hitesh Bhudia:169646

Supervisor: Mr. Tiberius Tabulu

An Informatics and Computer Science Project Proposal Submitted to the School of
Computing and Engineering Sciences in partial fulfillment of the requirements for the
award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Declaration and Approval

The student declares that this project proposal has not been submitted to Strathmore University or any other University for the award of a Degree in Bachelor of Science in Informatics and Computer Science or any other Degree. To the best of the student's knowledge and belief, the research proposal contains no material previously published or written by another person except where due reference is made in the research proposal itself.

Student Admission Number: 169646

Sign: _____

Date: _____

Supervisor's Signature:

Sign: _____

Date: _____

Acknowledgment

The researcher would like to extend heartfelt gratitude to the assigned supervisor, Mr. Tiberius Tabulu, for the unwavering support, technical guidance, and expert mentorship provided throughout the conceptualization and formulation of this research proposal.

Abstract

University campus networks support learning platforms, research services and multimedia applications whose changing traffic demands can overload particular links while alternative paths remain underused. Congestion-aware routing offers a means of redistributing this traffic, although frequent routing changes can impair operational stability. This study investigates a Neuro-Adaptive Network Flow Orchestrator that combines deep reinforcement learning with software-defined networking for proactive congestion control. The objectives are to examine congestion-management challenges, compare existing solutions, identify their limitations, develop a guarded routing prototype, and validate its performance. The experimental approach uses measured telemetry, a Proximal Policy Optimization model and an administrator interface. The prototype incorporates network emulation, candidate-route selection, operational safeguards and execution records. Further evaluation will compare the learned policy with conventional and non-learning routing methods under matched normal, burst and sustained-load conditions. Goodput, round-trip time, packet delivery, link utilization, congestion duration and route-change frequency will be measured, with separate training, validation and test data. The anticipated contribution is a reproducible evaluation of whether guarded adaptive routing improves traffic delivery and permits timely congestion intervention within a bounded campus-network environment.

Keywords: Deep reinforcement learning; Software-defined networking; Proactive congestion control; Network emulation; Routing stability

Table of Contents

Declaration and Approval.....	ii
Acknowledgment.....	iii
Abstract.....	iv
Table of Contents.....	v
List of Figures.....	ix
List of Tables.....	x
List of Abbreviations.....	xi
List of Equations.....	xiii
Chapter 1: Introduction.....	1
1.1 Background Information.....	1
1.2 Problem Statement.....	2
1.3 Research Objectives.....	3
1.3.1 General Objective.....	3
1.3.2 Specific Objectives.....	3
1.4 Research Questions.....	4
1.5 Justification.....	4
1.6 Scope.....	5
1.7 Limitations and Delimitations.....	5
1.7.1 Limitations.....	5
1.7.2 Delimitations.....	6
Chapter 2: Literature Review.....	7
2.1 Introduction.....	7
2.2 Current State of Campus-Network Congestion Management.....	7
2.2.1 Routing and Traffic-Engineering Approaches.....	7
2.2.2 Congestion Indicators.....	8
2.2.3 Measurement and Operational Challenges.....	9
2.3 Related Applications and Research Implementations.....	9
2.3.1 Cisco Catalyst Center.....	9
2.3.2 Juniper Mist and Marvis AI Assistant.....	11
2.3.3 Arista CloudVision.....	12
2.3.4 DRL–GNN Routing Optimization.....	13

2.3.5 ENERO.....	14
2.3.6 RouteNet-Fermi.....	14
2.3.7 Graph Transformer Star Routing.....	15
2.3.8 Comparative Analysis.....	16
2.4 Gaps in Related Works.....	17
2.5 Conceptual Framework.....	18
Chapter 3: Development Methodology.....	21
3.1 Introduction.....	21
3.2 Research Paradigm.....	21
3.2.1 Data Acquisition.....	21
3.2.2 Data Processing.....	22
3.2.3 Model Training.....	23
3.2.4 Model Validation and Testing.....	24
3.3 Software Development Methodology.....	25
3.3.1 Justification of the Methodology.....	26
3.4 Methodology Diagram.....	26
3.4.1 Prototype Planning.....	28
3.4.2 Initial Prototype Development.....	28
3.4.3 Iterative Refinement.....	28
3.4.4 Testing.....	28
3.4.5 Final Prototype Evaluation.....	29
3.5 System Development Tools and Techniques.....	29
3.5.1 Network Emulation and Control.....	30
3.5.2 Learning and Decision Inspection.....	30
3.5.3 Operational Safeguards.....	31
3.6 System Deliverables.....	31
3.6.1 System Documentation.....	31
3.6.2 Authentication Module.....	32
3.6.3 Telemetry Module.....	32
3.6.4 Cognitive Routing and Safeguard Module.....	32
3.6.5 Administrator Dashboard and Reports.....	32
Chapter 4: System Analysis and Design.....	33
4.1 Introduction.....	33

4.2 System Requirements.....	33
4.2.1 Functional Requirements.....	34
4.2.2 Non-Functional Requirements.....	37
4.2.3 Data Requirements.....	39
4.2.4 Interface and Integration Requirements.....	41
4.2.5 Requirements Traceability.....	42
4.3 System Analysis Diagrams.....	43
4.3.1 System Boundary and Actors.....	43
4.3.2 Observation-to-Assessment Workflow.....	46
4.3.3 Proposal, Approval and Execution Workflow.....	48
4.3.4 Cancellation, Override and Failure Behaviour.....	50
4.3.5 Information and State Analysis.....	52
4.4 System Design Diagrams.....	52
4.4.1 System Architecture and Class Design.....	52
4.4.2 Logical Data Design.....	56
4.4.3 Interface and Event Design.....	61
4.4.4 Learning Model and Safeguard Design.....	63
4.4.5 Operator Interaction Design.....	65
4.4.6 Deployment and Recovery Design.....	67
Chapter 5: System Implementation, Testing and Evaluation.....	70
5.1 Introduction.....	70
5.2 Implementation Environment.....	70
5.2.1 Hardware and Deployment Environment.....	70
5.2.2 Software, Services and Configuration.....	73
5.2.3 Deployment and Reproducibility.....	75
5.3 System Implementation.....	76
5.3.1 Identity, Resource Scope and Application Data.....	76
5.3.2 Emulated Devices and Measurement Acquisition.....	77
5.3.3 Communication, Storage and Display.....	77
5.3.3.1 Snapshot and Message Format.....	77
5.3.3.2 Identity and Trust at the Device Boundary.....	78
5.3.3.3 Rate Derivation and Persistence.....	78
5.3.3.4 Buffering, Retry and Reconnection.....	79

5.3.3.5 Disconnection and Power-Recovery Boundaries.....	80
5.3.4 Numerical Simulation and Controlled Network Actions.....	80
5.3.5 Model Training, Selection and Diagnostic Inference.....	81
5.3.5.1 Observation and Policy Implementation.....	81
5.3.5.2 Training and Selection Chronology.....	82
5.3.5.3 Frozen Inference and Operational Integration.....	83
5.3.6 Alerts, Reports and Operator Evidence.....	83
5.4 Testing and Evaluation.....	84
5.4.1 Test Strategy and Acceptance Criteria.....	84
5.4.2 Test Cases, Defects and Retesting.....	85
5.4.3 Project-Specific Evaluation.....	95
5.4.3.1 Held-Out Routing Performance.....	95
5.4.3.2 Training Behaviour and Stability.....	99
5.4.3.3 Inference, Control and Observation Timing.....	100
5.4.3.4 Application and Recovery Evaluation.....	101
5.4.4 Results Presentation and Interpretation.....	102
5.5 Version Control and Implementation Evidence.....	102
5.6 Evaluation Results, Discussion and Limitations.....	105
5.6.1 Objective i: Congestion Indicators and Traffic-Management Challenges.....	105
5.6.2 Objective ii: Existing Intelligent Network-Management Solutions.....	105
5.6.3 Objective iii: Limitations in Timing, Stability and Reproducibility.....	105
5.6.4 Objective iv: Design and Development of the Integrated Prototype.....	106
5.6.5 Objective v: Comparative Performance Validation.....	106
5.6.6 Validity and Generalization Limits.....	107
References.....	109
Appendix 1: Gantt Chart.....	112

List of Figures

Figure 2.1. Cisco Catalyst Center network-health dashboard.....	10
Figure 2.2. Marvis AI Assistant port-flap analysis.....	12
Figure 2.3. Arista CloudVision network operating model.....	13
Figure 2.4. Conceptual framework diagram.....	20
Figure 3.1. Agile prototyping lifecycle.....	27
Figure 4.1. Application actors and use cases.....	45
Figure 4.2. Observation validation and assessment activity.....	47
Figure 4.3. Approval and verified manual execution.....	49
Figure 4.4. Timed-override restoration and guarded return.....	51
Figure 4.5. System architecture and module integration.....	54
Figure 4.6. Command execution classes and responsibilities.....	55
Figure 4.7. Logical identity, scope and observation schema.....	58
Figure 4.8. Logical execution, publication and control schema.....	60
Figure 4.9. Authorized realtime reconnection and reconciliation.....	62
Figure 4.10. Frozen-model inference and autonomous admission design.....	64
Figure 4.11. Operator evidence-view wireframe.....	66
Figure 4.12. Deployment and laboratory file-handoff boundaries.....	68
Figure A1. Original project time schedule.....	112

List of Tables

Table 2.1: Comparative analysis of reviewed systems.....	16
Table 3.1: Development tools and their roles.....	29
Table 4.1: Functional requirements.....	34
Table 4.2: Non-functional requirements.....	37
Table 4.3: Objective-to-requirement traceability.....	42
Table 4.4: Logical data entities and relationships.....	56
Table 5.1: Hardware, runtime and deployment conditions.....	71
Table 5.2: Software versions and implementation roles.....	73
Table 5.6: PPO-minus-baseline paired differences and 95% confidence intervals.....	97
Table 5.8: Recorded inference and control timing.....	100
Table 5.9: Implementation and evaluation evidence register.....	103

List of Abbreviations

ADR – Architecture Decision Record
AI – Artificial Intelligence
API – Application Programming Interface
CI – Confidence Interval
CPU – Central Processing Unit
CSV – Comma-Separated Values
DQN – Deep Q-Network
DRL – Deep Reinforcement Learning
DSCP – Differentiated Services Code Point
ECMP – Equal-Cost Multipath
FRR – FRRouting
GNN – Graph Neural Network
GTSR – Graph Transformer Star Routing
HTB – Hierarchical Token Bucket
ICMP – Internet Control Message Protocol
ICT – Information and Communication Technology
IP – Internet Protocol
JSON – JavaScript Object Notation
LAN – Local Area Network
LLDP – Link Layer Discovery Protocol
MAC – Media Access Control
ML – Machine Learning
NANFO – Neuro-Adaptive Network Flow Orchestrator
OOAD – Object-Oriented Analysis and Design
OSPF – Open Shortest Path First
OWASP – Open Worldwide Application Security Project
PDF – Portable Document Format
PPO – Proximal Policy Optimization
QoS – Quality of Service
REST – Representational State Transfer
RTT – Round-Trip Time

SDN – Software-Defined Networking
SQL – Structured Query Language
TCP – Transmission Control Protocol
UAT – User Acceptance Testing
UDP – User Datagram Protocol
UUID – Universally Unique Identifier
WAN – Wide Area Network
WSTG – Web Security Testing Guide

List of Equations

Equation 3.1: Feature transformation.....	22
Equation 3.2: Routing reward.....	23
Equation 3.3: PPO clipped policy objective.....	23
Equation 3.4: Conditional queue envelope.....	31
Equation 3.5: Queue-function drift condition.....	31

Chapter 1: Introduction

1.1 Background Information

A campus network can have spare capacity and still deliver a poor user experience when traffic concentrates on a small number of links. Traffic engineering addresses this imbalance by measuring demand and directing flows through available resources rather than treating connectivity alone as sufficient evidence of network performance (Farrel, 2024). In higher education, this distinction matters because teaching, research and administration increasingly depend on shared digital services. The African Union Commission's (2022) Digital Education Strategy identifies network infrastructure as a foundation for digital learning, assessment and educational administration.

Campus traffic combines interactive communication with less delay-sensitive transfers. An online class, a research upload and a software download can compete for the same path despite having different delivery requirements. When incoming traffic exceeds the service available at an interface, packets accumulate in queues; sustained overload can increase delay and packet loss. The resulting degradation depends on traffic distribution, queue configuration and routing choices, so an increase in total capacity does not necessarily resolve an unfavourable distribution of traffic (Farrel, 2024; Ferriol-Galmés et al., 2023).

Conventional routing establishes reachability through distributed control. Open Shortest Path First (OSPF), for example, calculates shortest paths from configured link costs and a link-state database, and recalculates routes when the topology changes (Moy, 1998). However, nominal link costs need not reflect instantaneous queue pressure or offered demand. Software-Defined Networking (SDN) enables a controller to collect network information and coordinate programmable forwarding decisions, creating opportunities for state-dependent traffic engineering (Farrel, 2024).

Deep Reinforcement Learning (DRL) extends this approach by learning how routing actions affect a performance objective. Recent work has combined learning with graph representations, local search and topology-aware packet routing. ENERO demonstrated the value of combining a learned routing proposal with conventional optimization, while Graph Transformer Star Routing examined adaptation to

previously unseen link failures (Almasan, Xiao, et al., 2022; Li et al., 2025). These developments suggest that learned routing can respond to conditions that configured shortest paths do not directly represent.

The remaining challenge is to connect useful adaptation with stable and timely network operation. A policy may improve average delivery while making excessive path changes, and an improvement observed after congestion occurs does not establish preventive intervention. Furthermore, network-performance predictions can contain model error, while reinforcement-learning comparisons can be sensitive to the number of training runs and evaluation conditions (Agarwal et al., 2021; Ferriol-Galmés et al., 2023). These issues make operational safeguards and controlled evaluation important parts of a deployable research prototype.

The Neuro-Adaptive Network Flow Orchestrator (NANFO) will investigate whether a compact DRL policy using Proximal Policy Optimization (PPO), supported by measured telemetry and explicit route-change restrictions, can improve congestion management in an emulated campus network. The study will examine existing solutions, identify the relevant integration gaps and evaluate a guarded prototype against defined baselines. Its guiding proposition is that congestion-aware route selection can improve delivery when alternative capacity is available, while operational restrictions can reduce unnecessary changes. The research questions will test this proposition and the timing of interventions under controlled workloads.

1.2 Problem Statement

A campus network should distribute traffic across available paths while maintaining responsive communication and predictable forwarding. Its management system should identify developing congestion, select an appropriate intervention and verify the outcome without unnecessary routing changes (Farrel, 2024).

In practice, routing based on nominal OSPF costs does not continuously incorporate queue occupancy and changing traffic demand into path selection (Moy, 1998). Learned routing systems address aspects of this shortfall: ENERO optimizes wide-area traffic distribution, and Graph Transformer Star Routing improves packet-routing performance under evaluated topology changes (Almasan, Xiao, et al., 2022; Li et al., 2025). However, their different action spaces, traffic models and experimental

environments leave unresolved how a compact policy should combine early intervention, route-change restrictions and verified execution in a bounded campus-network setting. Network modelling also introduces prediction error that must be considered before a proposed change is accepted (Ferriol-Galmés et al., 2023).

This gap matters because overloaded paths can degrade teaching and research services, while excessive reconfiguration can offset the benefit of adaptive routing. The study will develop and evaluate NANFO to connect measured congestion indicators, DRL-based path selection and operational safeguards. Matched experiments will determine whether the prototype improves delivery and controls route changes, and whether verified actions precede defined congestion events. The evaluation will provide evidence for the tested emulated environment rather than presume effectiveness on a live university network.

1.3 Research Objectives

1.3.1 General Objective

To develop a stability-aware deep reinforcement learning flow orchestrator for proactive congestion control in an emulated software-defined campus network.

1.3.2 Specific Objectives

- i. To analyse congestion indicators and traffic-management challenges under normal, burst and sustained-load campus-network conditions by September 2026.
- ii. To compare the approaches and reported contributions of at least three existing intelligent network-management solutions by September 2026.
- iii. To identify limitations in existing solutions concerning intervention timing, routing stability and reproducible evaluation by September 2026.
- iv. To design and develop a PPO-based flow orchestrator with telemetry, operational safeguards and an administrator interface by October 2026.
- v. To test and validate the orchestrator against OSPF and non-learning baselines using delivery, congestion and routing-stability metrics by November 2026.

1.4 Research Questions

- i. What congestion indicators and traffic-management challenges arise under normal, burst and sustained-load campus-network conditions?
- ii. What approaches and reported contributions distinguish existing intelligent network-management solutions?
- iii. What limitations remain in existing solutions concerning intervention timing, routing stability and reproducible evaluation?
- iv. How can a PPO-based flow orchestrator integrate telemetry, operational safeguards and an administrator interface?
- v. How does the orchestrator perform against OSPF and non-learning baselines in delivery, congestion and routing stability?

1.5 Justification

Existing research establishes that network state can inform traffic allocation and that learned models can support routing optimization. ENERO demonstrates a combination of DRL and local search, while RouteNet-Fermi predicts delay, jitter and loss across varied network configurations (Almasan, Xiao, et al., 2022; Ferriol-Galmés et al., 2023). Commercial platforms also connect telemetry with troubleshooting and change management (Arista Networks, 2026; Cisco Systems, 2026; Juniper Networks, 2025). These contributions provide both technical foundations and practical requirements for NANFO.

The proposed study addresses the connection between a routing decision and its operational consequences. Improvements in a learning objective are insufficient if the resulting policy produces unnecessary reconfiguration or acts only after a service has deteriorated. Assessing intervention timing, route changes and verified execution alongside delivery metrics will therefore provide a more useful basis for judging the prototype than average throughput alone. Explicit model and observation assumptions will also support interpretation of its safeguards (Brunke et al., 2022).

The immediate beneficiaries are network administrators and researchers who require an inspectable environment for comparing congestion-management strategies. Administrators can examine observations, proposed routes and execution evidence

through one interface, while researchers can reproduce the workload and baseline comparison. The study contributes a controlled evaluation of guarded campus-style traffic engineering; better use of available paths is the intended practical benefit, while the evidence will inform subsequent deployment decisions.

1.6 Scope

The technical scope covers a wired campus-style topology containing five forwarding nodes and four hosts, with two candidate paths for a foreground flow. The operator environment uses Mininet, Open vSwitch and Ryu, and the matched routing experiment uses Linux forwarding with FRRouting. The PyTorch model implements categorical PPO. The administrator application provides telemetry, topology and digital-twin views, simulation, intent review, controlled execution, audit records and reports.

The data scope comprises synthetic workload schedules and measurements generated within the isolated emulated network. Observations include link utilization, queue occupancy, round-trip time, useful received throughput or goodput, and packet loss. Normal-load, burst and sustained-load scenarios will be evaluated using recorded capacities, measurement intervals and workload seeds. Existing experiment records provide development evidence; the study does not rely on undisclosed institutional traffic logs.

The study targets completion by November 2026. Literature and requirements refinement will conclude in September, integration and safeguard refinement in October, and evaluation, documentation and presentation preparation in November. The original project schedule is reproduced in Appendix 1.

1.7 Limitations and Delimitations

1.7.1 Limitations

The available development hardware limits the scale and duration of training and emulation. Shared-host scheduling can affect measured timing, and synthetic workloads may not reproduce the full variability of operational campus traffic. Access to verified institutional topology and flow records is also limited, reducing the extent to which the results can represent a particular university. The existing selected-model

evaluation covers one trained initialization; additional runs are needed to assess sensitivity to training randomness.

The learning benchmark and the operator testbed use different control interfaces. Integration therefore depends on compatible observations, action mappings and calibrated bounds. A deterministic network model cannot represent every packet-level effect, and errors in arrival or service bounds may affect a conditional safety assessment. These constraints will be considered when interpreting the prototype’s performance and intervention results.

1.7.2 Delimitations

The study intentionally focuses on wired Ethernet and Internet Protocol (IP) traffic engineering, two-path selection and an isolated emulator. Wireless radio optimization, cellular slicing, intrusion-detection research, custom switch hardware and production deployment fall outside this boundary. The learned policy will select routes; manually approved shaping and policing remain separate operator functions. Physical power control and third-party plugin execution are excluded from the research evaluation. These choices keep the experiment focused on congestion, route stability and verified intervention.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews traffic engineering and intelligent network management to establish the basis for NANFO. Section 2.2 examines the current state of congestion management and its measurement challenges. Section 2.3 compares existing applications and research implementations, and Section 2.4 synthesizes their unresolved limitations. Section 2.5 presents the conceptual framework linking those findings to system development and evaluation.

The review prioritizes peer-reviewed studies and official publications from 2021–2026, with emphasis on recent work concerning network modelling, learned routing and operational management. Sources were selected for direct relevance, identifiable experimental methods and sufficient evidence to distinguish reported findings from proposed capabilities. Protocol and original algorithm references are used where their definitions remain necessary. Vendor publications establish product functionality, while peer-reviewed experiments support analysis of performance, methodology and generalization.

2.2 Current State of Campus-Network Congestion Management

2.2.1 Routing and Traffic-Engineering Approaches

Traffic engineering seeks to improve network performance through measurement, modelling and control of traffic distribution. Farrel (2024) distinguishes proactive actions intended to prevent unfavourable network states from reactive actions responding to existing events. This distinction makes intervention timing central to congestion control: an adaptive route change can be useful even when it is reactive, but prevention requires evidence that the action occurred before the relevant congestion condition.

Traditional link-state routing provides a dependable mechanism for maintaining reachability. OSPF recalculates shortest paths following topology changes and supports equal-cost alternatives, but its path selection follows configured costs rather than a continuously learned representation of demand (Moy, 1998). State-dependent traffic engineering adds measurements such as utilization and queue pressure to this

decision process. SDN makes such coordination feasible through programmable forwarding, although the controller still requires a policy for choosing and timing an action (Farrel, 2024).

Learning-based systems differ in what they optimize. Some estimate network performance, while others allocate demands or alter forwarding decisions. Ferriol-Galmés et al. (2023) model the relationship between queues, traffic and path performance; Almasan, Xiao, et al. (2022) combine learned proposals with local search; and Li et al. (2025) use topology-aware multi-agent routing. These approaches are complementary rather than interchangeable. Their relevance to a campus network depends on the available observations, action granularity and time required to apply a decision.

2.2.2 Congestion Indicators

Link utilization expresses a measured transmission rate relative to link capacity. It indicates how heavily a resource is being used, but interpretation depends on the sampling interval and traffic distribution. A long averaging interval may conceal a short burst, while a highly utilized link may still meet its service requirements. Traffic-engineering analysis consequently needs both time-varying measurements and information about the affected flows (Farrel, 2024).

Queue occupancy captures traffic waiting for transmission. It provides information about the imbalance between arrival and service rates that a utilization value alone may miss. Queue size, scheduling discipline and traffic burstiness also influence delay and loss. RouteNet-Fermi explicitly represents these relationships, demonstrating that performance modelling benefits from considering queues and paths together rather than treating links as independent resources (Ferriol-Galmés et al., 2023).

Round-trip time, goodput and packet loss describe delivery outcomes. Round-trip time includes forward-path, return-path and endpoint effects; goodput measures useful payload received; and packet-delivery ratio relates received packets to transmitted packets over the defined observation interval. These quantities must retain their measurement boundaries. A sender's intended rate is not received throughput, and a failed latency probe does not provide a measured delay. Combining these outcomes

with utilization and queue occupancy provides a clearer account of congestion than any single average (Farrel, 2024; Ferriol-Galmés et al., 2023).

2.2.3 Measurement and Operational Challenges

The first challenge is observation quality. Polling intervals, counter resets, missing readings and the location of a measured queue can influence the state presented to a controller. Short experiments are also sensitive to packets remaining in flight when measurement stops. The collection process therefore needs consistent timestamps, units and packet-accounting boundaries. These requirements follow from the importance of accurate performance measurement in traffic engineering (Farrel, 2024).

A second challenge is the difference between prediction and control. RouteNet-Fermi achieved useful prediction accuracy across several test settings, but its reported error changed with the source of the observations (Ferriol-Galmés et al., 2023). An optimizer using a model must account for this uncertainty when evaluating a candidate action. A visual representation of topology alone does not establish the accuracy of predicted behaviour.

A third challenge concerns adaptation under uncertainty. Frequent route changes can increase control activity and reduce forwarding predictability. A reward that discourages switching expresses a preference, whereas an execution restriction can prevent a disallowed change. Safe-learning research distinguishes these roles and emphasizes that constraint satisfaction depends on the system model and assumptions (Brunke et al., 2022). For a campus-style controller, the practical question is how to balance useful responsiveness with sufficient path-holding time and verifiable execution.

2.3 Related Applications and Research Implementations

2.3.1 Cisco Catalyst Center

Cisco Catalyst Center, formerly Cisco DNA Center, provides enterprise discovery, provisioning, policy automation and assurance. Its assurance functions relate device, client and application observations to network health, while application programming interfaces (APIs) and event notifications support integration with operational tools.

The 2.3.7 data sheet also documents third-party device reachability and topology visibility (Cisco Systems, 2026). These capabilities make it relevant to campus administrators investigating service degradation across several devices.

Its main contribution to NANFO is the link between telemetry and an actionable operator workflow. However, visibility does not imply equivalent provisioning support for every device, and functionality depends on the documented compatibility and subscription arrangements. The vendor data sheet describes capabilities rather than an independently controlled routing comparison. It supports requirements for visibility and management integration, but does not establish the effect of a particular learned policy on campus route-change frequency. Figure 2.1 shows the network-health dashboard documented in the Catalyst Assurance user guide (Cisco Systems, 2025).

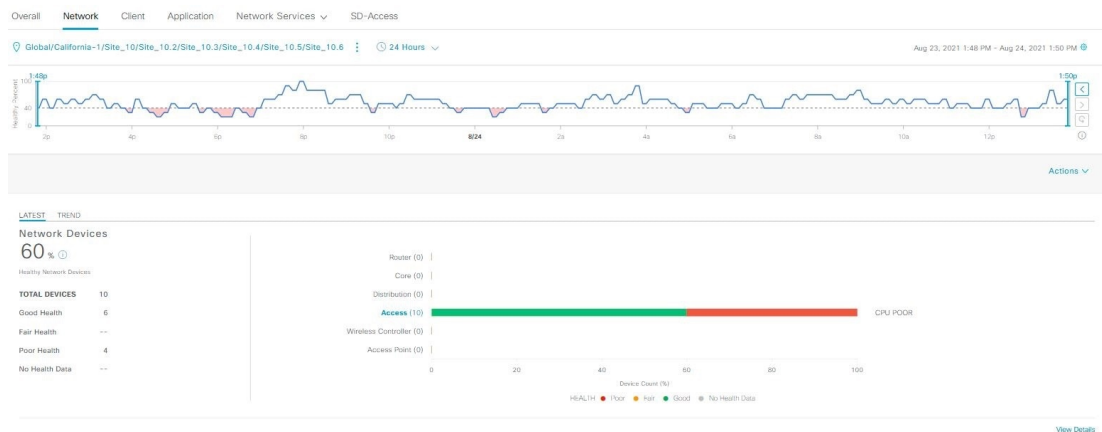


Figure 2.1. Cisco Catalyst Center network-health dashboard

Note. From *Cisco Catalyst Assurance user guide, release 2.3.7.x: Monitor and troubleshoot network health*, Figure 1, by Cisco Systems (2025). Copyright Cisco Systems, Inc. Source URL is provided in the reference list.

The dashboard combines a time-based health trend with device categories and health summaries. This allows an operator to relate a period of degraded service to the affected resources. NANFO applies the same principle by linking congestion observations to identifiable paths and reviewable proposed actions. The screenshot illustrates Cisco's interface rather than a measurement from the NANFO experiment.

2.3.2 Juniper Mist and Marvis AI Assistant

Marvis AI Assistant provides conversational access to network information and supports troubleshooting across the Mist environment. Juniper Networks (2025) describes a client-to-cloud operational view and Marvis Actions, which prioritizes high-impact issues and supports remediation workflows. Its conversational interface can connect an administrator's question to device observations and historical trends.

The system demonstrates the value of presenting a problem in terms meaningful to an administrator, such as an affected client, application or site. Its limitation as evidence for this study is that the product account does not disclose a reproducible two-path learning experiment or a controlled measurement of routing stability. A vendor-authored account also serves a different purpose from an independent comparison that reports unsuccessful cases. NANFO adopts the principle of evidence-linked interaction while evaluating its routing policy independently. Figure 2.2 shows a documented Marvis response to a request about port-flap trends.

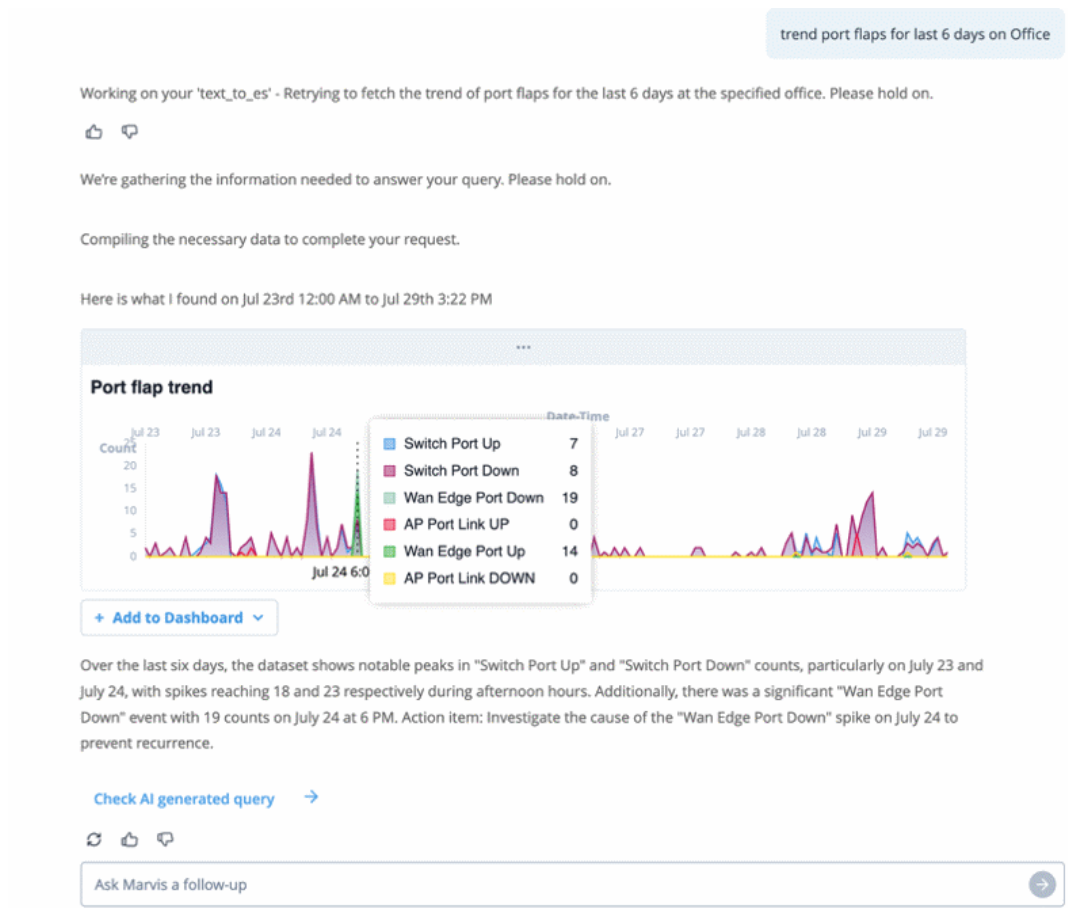


Figure 2.2. Marvis AI Assistant port-flap analysis

Note. From *Marvis AI assistant datasheet*, Figure 1, by Juniper Networks (2025).

Copyright 2025 by Juniper Networks, Inc. Source URL is provided in the reference list.

The response presents a time-series chart and an interpretation of the observed port events. Its relevance is the connection between a question, supporting observations and an explanation that an operator can inspect. NANFO will provide measured telemetry and action reasons through its interface; a conversational assistant is outside the implemented research scope.

2.3.3 Arista CloudVision

CloudVision provides streaming telemetry, state history, provisioning and change-management capabilities across data-centre, campus, branch and wide-area environments. Arista Networks (2026) documents both on-premises and software-as-a-service deployment. Its network-wide state history is particularly relevant when an administrator needs to relate a configuration change to subsequent behaviour.

CloudVision’s contribution lies in connecting intended configuration, applied state and operational observations. This is important because a successful request does not necessarily demonstrate that a network change took effect. Nevertheless, the product data sheet is not a controlled evaluation of learned congestion mitigation. Its functionality and licensing also depend on the selected deployment and devices. It provides a reference for change assurance rather than an experimentally equivalent routing baseline. Figure 2.3 reproduces the operating model in Arista's data sheet.

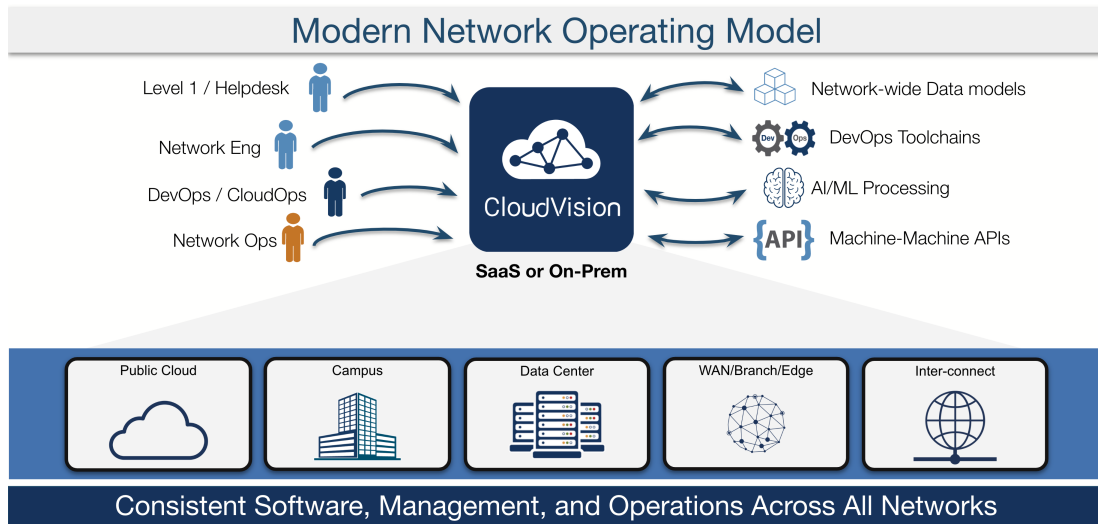


Figure 2.3. Arista CloudVision network operating model

Note. From *CloudVision data sheet* (p. 1), by Arista Networks (2026). Copyright 2025 by Arista Networks, Inc. Source URL is provided in the reference list.

The diagram connects operational roles with network-wide data models, automation tools and APIs across campus and other network domains. The shared management layer explains why a change-management platform is more than a topology viewer. For NANFO, the relevant design principle is to connect the operator's decision with the applied configuration and subsequent readback evidence.

2.3.4 DRL–GNN Routing Optimization

Almasan, Suárez-Varela, et al. (2022) combined a Graph Neural Network (GNN) with a Deep Q-Network (DQN) to allocate traffic demands in an optical transport network. The agent represented link capacity and candidate actions in a graph and selected among a bounded set of paths. Its objective was to maximize allocated traffic over an

episode. Evaluation on 180 unseen synthetic topologies and 232 unseen real-world topologies provided substantial evidence of generalization beyond the training graph.

The study’s strength is its explicit treatment of topology variation and its extensive evaluation set. However, accepted demands could not be rerouted or split and remained allocated until the episode ended. These assumptions simplify aspects of temporal forwarding behaviour that matter when an active campus flow is repeatedly redirected. The findings justify studying bounded candidate-path decisions, but the optical allocation results do not establish the delay or switching behaviour of an active-flow controller. NANFO must therefore evaluate route changes as a separate outcome.

2.3.5 ENERO

ENERO combines a graph-based DRL policy with local search for wide-area routing optimization. The learned stage provides an initial routing configuration, after which conventional search improves the solution. Almasan, Xiao, et al. (2022) reported a mean optimization time of 4.5 seconds for evaluated topologies with up to 100 links. Focusing on selected critical demands reduced the size of the optimization problem while preserving useful traffic-engineering decisions.

The hybrid design provides an important alternative to a learning-only explanation of performance. Its results suggest that useful optimization can emerge from combining a learned proposal with a simpler search procedure. This strengthens the case for including a load-aware heuristic in NANFO’s evaluation rather than comparing only with nominal-cost routing. At the same time, ENERO’s traffic matrices and routing operations differ from the proposed campus experiment. Its optimization time describes a particular processing task; the end-to-end time for observation, authorization, installation and verification requires separate measurement.

2.3.6 RouteNet-Fermi

Ferriol-Galmés et al. (2023) developed RouteNet-Fermi to predict delay, jitter and loss from network topology, traffic, routing and queue configuration. The model uses graph-based relationships among paths, links and queues. Its evaluation included varied traffic profiles, networks of up to 300 nodes, real traffic traces and a physical

testbed. On a 1,000-sample large-network test set, the authors reported a mean relative delay error of 6.24%.

The combination of simulation, trace-based and physical-testbed evaluation is a methodological strength. It provides stronger evidence of modelling capability than evaluation on one synthetic topology alone. However, the reported error varied across settings: the study reported approximately 5.64% with real-network traces and 11% in the physical testbed. These differences show that useful generalization does not eliminate measurement and distribution effects (Ferriol-Galmés et al., 2023).

RouteNet-Fermi predicts performance rather than directly selecting and authorizing a routing action. Its relevance to NANFO is therefore the treatment of model error and the separation of predicted from measured behaviour. A simulation-informed routing proposal still requires compatible observations and execution verification. This distinction also prevents a favourable modelling result from being interpreted as proof of safe closed-loop control.

2.3.7 Graph Transformer Star Routing

Li et al. (2025) introduced Graph Transformer Star Routing (GTSR), combining multi-agent PPO, graph-transformer message passing and path-based readout. A virtual star node broadens communication among graph representations. The implementation used PyTorch and OMNeT++ to investigate packet routing on Nsfnet and other Topology Zoo networks, including tests on unseen link-failure configurations after training.

The experiments specified 1 Mbps links, 2 ms link delay, 512-byte packets and a two-second control step, with emphasis on heavy uniform traffic. Training used 36,000 steps, and testing on each faulty topology used frozen model parameters. Comparisons included equal-cost multipath routing, other learned approaches and reduced versions of the proposed architecture. The authors reported lower delay and loss across the evaluated cases, while the ablations helped explain the contribution of the graph-transformer and readout components (Li et al., 2025).

The explicit experimental settings and component comparisons strengthen the interpretation of the results. However, uniformly generated simulated traffic differs from the mixed demand and burst timing of the proposed campus experiment. Multi-

agent topology robustness also does not establish the benefit of a compact two-action PPO model. GTSR supports the use of held-out topology tests and ablation, while leaving NANFO’s route stability, controller overhead and early-intervention behaviour to be measured in its own environment.

2.3.8 Comparative Analysis

The reviewed systems address three related themes: operational assurance, routing optimization and performance modelling. The commercial platforms emphasize operator visibility and controlled change, whereas the research implementations expose specific algorithms and evaluation assumptions. Table 2.1 compares their contributions and the limitations relevant to NANFO.

Table 2.1: Comparative analysis of reviewed systems

System and source	Strength and evidence	Limitation relevant to NANFO
Catalyst Center (Cisco Systems, 2026)	Assurance, provisioning and third-party visibility.	Vendor capability account; no equivalent open routing experiment.
Marvis AI Assistant (Juniper Networks, 2025)	Evidence-linked troubleshooting and prioritized remediation.	Undisclosed learning/evaluation details for the proposed routing task.
CloudVision (Arista Networks, 2026)	State history and change assurance, including campus deployments.	Device/deployment dependencies; no matched learned-routing result.
DRL–GNN (Almasan, Suárez-Varela, et al., 2022)	Optical allocation tested on 412 unseen topologies.	Accepted demands cannot be rerouted; route churn is outside that action model.
ENERO (Almasan, Xiao, et al., 2022)	DRL plus local search; 4.5-second mean optimization in tested networks.	WAN traffic/action model; computation time excludes NANFO’s full control path.

System and source	Strength and evidence	Limitation relevant to NANFO
RouteNet-Fermi (Ferriol-Galmés et al., 2023)	Delay, jitter and loss prediction; simulation and physical evidence.	Prediction error varies by setting; model output is not verified actuation.
GTSR (Li et al., 2025)	Multi-agent PPO; topology-change tests and architecture ablations.	Uniform simulated traffic; stability under campus bursts remains untested here.

The comparison also reveals why a single performance ranking would be misleading. Allocated optical bandwidth, prediction error, optimizer runtime and packet-delivery performance describe different tasks. A suitable evaluation must compare equivalent actions and workloads rather than transfer percentages between these studies.

2.4 Gaps in Related Works

The first gap concerns the continuity between measurement, decision and verified action. The commercial systems in Sections 2.3.1–2.3.3 connect observations with operational workflows, while the research implementations in Sections 2.3.4–2.3.7 concentrate on particular modelling or optimization problems. As Table 2.1 shows, these contributions do not provide a directly reproducible evaluation of the complete combination proposed for NANFO. The study therefore requires a traceable path from an observed congestion indicator to a candidate route, an accepted command and its measured outcome (Arista Networks, 2026; Cisco Systems, 2026; Li et al., 2025).

A second gap concerns routing stability during adaptation. The DRL–GNN study avoids rerouting previously accepted demands, while ENERO evaluates a different optimization process. These assumptions limit what their results establish about repeatedly changing an active campus flow (Almasan, Suárez-Varela, et al., 2022; Almasan, Xiao, et al., 2022). GTSR demonstrates topology adaptation, but its reported evaluation does not answer the proposed trade-off between path-holding restrictions and delivery under burst traffic. NANFO will address this through separate route-change metrics and comparison of the same policy with and without additional switching restrictions.

The third gap is evidence of proactivity. Proactive traffic engineering aims to prevent an unfavourable state rather than only correct it after occurrence (Farrel, 2024). Low average delay or loss does not identify when an action occurred relative to congestion onset. The reviewed performance results consequently motivate timestamped analysis of warning observations, verified actions and subsequent congestion events. Matched no-intervention runs will provide a reference when an action may prevent an event that otherwise would have occurred.

A fourth gap concerns experimental comparability and uncertainty. RouteNet-Fermi demonstrates that errors change between simulation and physical settings (Ferriol-Galmés et al., 2023), while Agarwal et al. (2021) show that a small number of learning runs can produce misleading conclusions when uncertainty is ignored. These findings support matched baselines, separate model-selection data and explicit reporting of sample sizes. They also challenge any assumption that a more complex learning model necessarily provides the most useful controller. Non-learning and hybrid alternatives remain necessary comparison points.

Together, these gaps establish the need to evaluate measurement, adaptive routing and operational control as a connected process. NANFO will combine measured telemetry, a compact PPO policy, route-change restrictions and an administrator workflow, then evaluate delivery, intervention timing and route stability under common experimental conditions. The conceptual framework in Section 2.5 expresses these relationships.

2.5 Conceptual Framework

The conceptual framework combines SDN telemetry, DRL-based decision-making and operational validation to investigate congestion control in a campus-style network. The network supplies measured inputs, the learning component proposes a route, and the validation stage determines whether the proposed action may proceed. Figure 2.4 shows the relationship among these stages and administrator oversight.

The input variables are link utilization, queue occupancy, round-trip time, goodput and packet loss. Topology and capacity describe the environment in which those observations arise. The processing variables include the PPO policy, route-change penalty, path-holding restrictions and conditional queue assessment. The output

variables are measured delivery performance, congestion duration, intervention lead time and route-change frequency. Routing method, workload and safeguard configuration are varied experimentally, while the comparison retains common topology and measurement rules.

The framework draws on state-dependent traffic engineering, learned sequential decision-making and model-based safety assessment (Brunke et al., 2022; Farrel, 2024; Schulman et al., 2017). It hypothesizes that directing traffic toward an available alternative path can improve delivery, while separate execution restrictions can reduce unnecessary changes. Model uncertainty and observation age mediate whether a candidate decision is accepted. Administrator review and recovery provide operational control over the experiment.

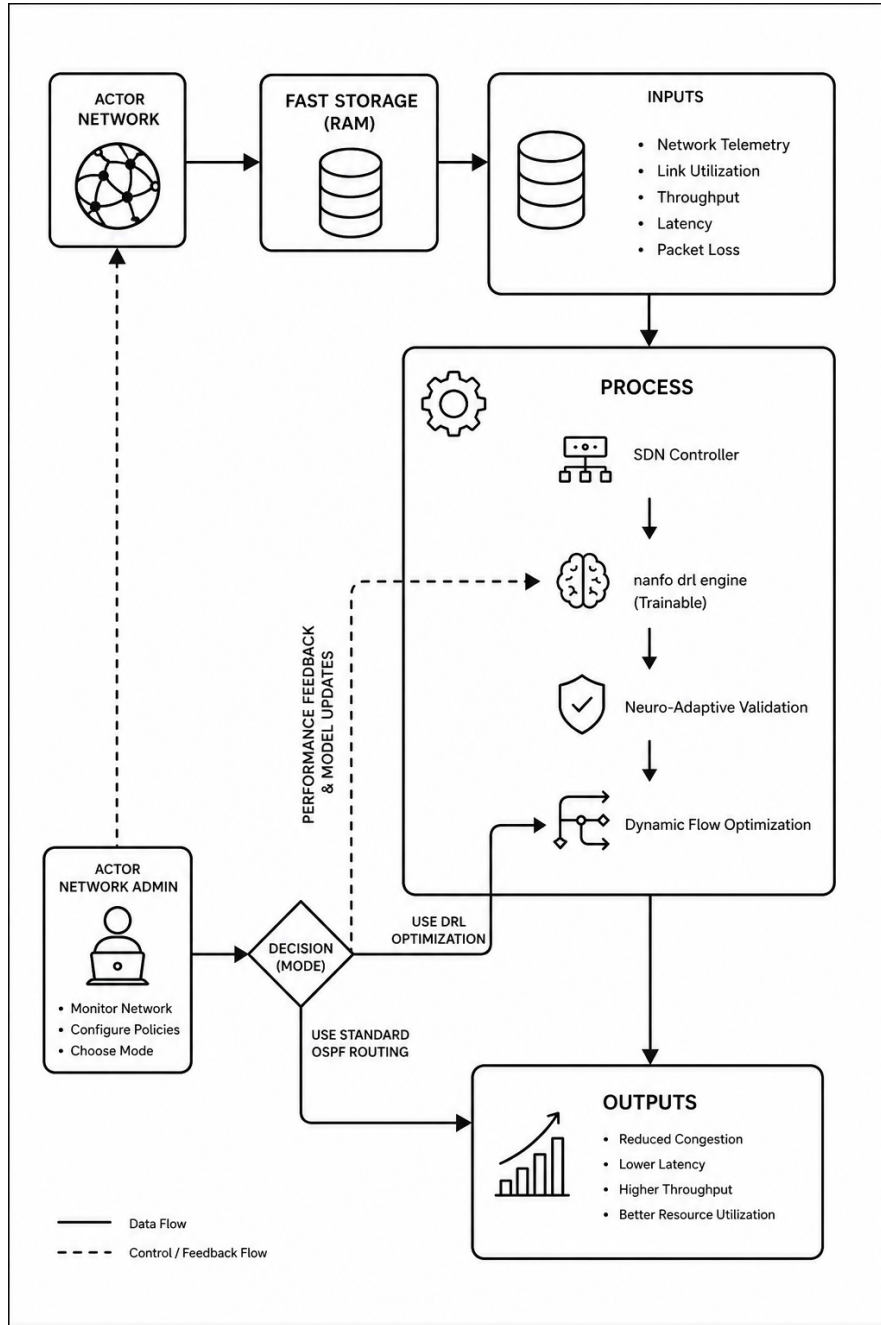


Figure 2.4. Conceptual framework diagram

The solid arrows show observations, processing and outcomes; dashed arrows represent control and feedback. The network actor denotes the emulated data plane, and the trainable engine denotes the separate PPO component. Training updates the model; evaluation uses frozen weights. Accepted and authorized routes proceed to installation and readback. Rejected proposals retain the permitted configuration, while measured outcomes test the expected relationships.

Chapter 3: Development Methodology

3.1 Introduction

This chapter presents the experimental research paradigm and Object-Oriented Analysis and Design (OOAD) approach used for NANFO. It describes data acquisition and processing, model training, validation, the Agile prototyping lifecycle, tools and deliverables. OOAD guides the modular application, while controlled experiments evaluate the learning policy and routing outcomes. Detailed system analysis and design will be presented in Chapter 4.

3.2 Research Paradigm

The study adopts an experimental, quantitative paradigm. Routing method, traffic scenario and safeguard configuration are the independent factors; delivery, congestion and route stability are the measured outcomes. Comparisons will retain common topology, offered demand, queue settings and observation rules. This design permits examination of whether a performance difference follows from the routing method rather than a different test environment.

Training, validation and final testing serve separate purposes. Training updates the model, validation selects candidates and settings, and reserved testing estimates performance on unseen workloads. This separation is important because repeated selection using final test outcomes can bias the reported comparison. Sample counts and uncertainty will be reported alongside point estimates, following the evaluation concerns identified by Agarwal et al. (2021).

3.2.1 Data Acquisition

The dataset consists of measurement records generated by synthetic traffic in an isolated campus-style network. The topology contains five forwarding nodes and four hosts, with two candidate paths between the foreground endpoints. The operator environment uses Mininet, Open vSwitch and Ryu. A separate matched Linux/FRRouting environment was developed for policy comparison so that OSPF and the learning methods use common forwarding and workload conditions. Versioned experiment records retain the configuration, raw observations and checkpoint identity.

Each record contains an experiment identifier, timestamps, measurement duration, path utilization, queue observations, endpoint sent and received counts, latency-probe results, offered rates, capacities and the verified previous route. Controller counters, Linux queue-discipline readback and endpoint probes provide complementary observations. Normal, burst and sustained-load traffic will vary demand relative to documented capacity. The collection procedure records the workload seed and configuration so that another method can be tested against the same schedule.

The retained training campaign collected 384 valid transitions, with 24 policy updates. The associated held-out benchmark used twelve workload seeds and four measured decisions per seed for each of five methods. These counts describe existing experiment records rather than the size of the planned traffic-family extension. Further collection will use a new, recorded split; institutional user identifiers and live campus packet contents are not required.

3.2.2 Data Processing

Processing checks record completeness, finite values, positive measurement durations and correspondence with the active experiment. Duplicate identifiers and inconsistent counter intervals are rejected. Queue and utilization values retain their original units and source. A missing required observation invalidates a learning transition; an unavailable round-trip-time measurement is represented by an explicit availability flag rather than a measured zero.

The implemented observation vector contains thirteen numerical features: utilization and queue occupancy for both paths; round-trip time, packet loss and goodput; requested foreground and background rates; elapsed time since the last route change; actual foreground rate; and both path capacities. One latency-availability indicator and two previous-route indicators produce sixteen model inputs. Fixed positive scales transform non-negative numerical values using Equation 3.1, where x_j is the observed value and s_j is its feature scale.

$$z_j = \ln(1 + x_j / s_j), \text{ where } s_j > 0$$

Equation 3.1: Feature transformation

The feature scales and action mapping are stored with the checkpoint. They are fixed independently of reserved test data. Related records remain within their declared

experiment partition. The current model uses one observed stationary frame; future workload phases and seed labels are excluded from its input. Class-balancing augmentation is unnecessary for this sequential interaction task. Coverage is instead controlled through the declared workload families and seed schedule.

3.2.3 Model Training

The routing task was formulated as a discrete-time sequential decision problem. The actor selects one of two candidate routes, and the critic estimates the return associated with the observed state. The PyTorch implementation uses separate thirty-two-unit hidden layers for the actor and critic. PPO was selected because its categorical policy matches the bounded action space and its clipped objective supports controlled policy updates (Schulman et al., 2017). PPO-based routing in Li et al. (2025) provides a recent application precedent, although NANFO uses a smaller architecture.

The reward combines received-payload efficiency with penalties for delay, loss, utilization, queue pressure and route changes. In Equation 3.2, G_t is goodput, F_t is actual positive offered foreground rate, D_t is round-trip time in milliseconds, L_t is loss fraction, U_t is maximum candidate-path utilization, Q_t is maximum candidate-path queue occupancy in packets, and C_t indicates a verified route change. The coefficients specify the implemented experiment configuration.

$$r_t = \min(G_t / F_t, 1) - 0.2(D_t / 50) - L_t \\ - 0.1U_t - 0.1(Q_t / 100) - 0.05C_t$$

Equation 3.2: Routing reward

A verified zero-reply outage retains an unavailable observed delay. The reward contract assigns a separately labelled 1,000 ms censored-delay penalty to that condition; the value is not reported as a latency measurement. Incomplete windows are excluded from training. Equation 3.3 defines the PPO policy objective, where ρ_t is the ratio of new and previous action probabilities, $\hat{A}_t$ is the estimated advantage and ϵ is the clipping parameter (Schulman et al., 2017).

$$L_{\text{clip}}(\theta) = \text{mean}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

Equation 3.3: PPO clipped policy objective

The selected recorded campaign used model seed 44, learning rate 0.003, discount factor 0.9 and sixteen-transition rollouts. Candidate checkpoints were assessed on

validation data before test access. Further refinement will retain independent workload partitions, bounded run budgets and an explicit stopping rule. Hyperparameter choices will be compared on validation runs; random cross-validation across adjacent transitions will not be used because it would mix dependent records. Saved weights, optimizer state, feature definitions and run configuration support reproducibility.

3.2.4 Model Validation and Testing

The evaluation compares PPO with nominal-cost OSPF, a load-aware heuristic and two constant-route policies. All methods use the same topology, addressing, offered workload, queue configuration and background-traffic placement. OSPF costs remain documented and unchanged during the primary comparison. The constant policies test whether an apparently useful learned agent merely prefers one path, while the heuristic provides a responsive non-learning alternative.

The retained twelve-seed benchmark provides a reproducible starting point. Further testing will use twelve fresh held-out workload seeds in each of the normal, burst and sustained-load families. This extends the earlier twelve-seed protocol to thirty-six workload cases per evaluated policy. Three independently trained initializations are planned to assess training variability. Each campaign will record the achieved counts, method order and exclusions. Model selection will use validation evidence, and the new final-test seeds will remain unavailable to training.

The primary outcomes are received-payload goodput and Internet Control Message Protocol (ICMP) round-trip time. Packet-delivery ratio, loss, maximum measured link utilization and congestion duration provide secondary delivery measures. Routing stability is assessed through verified changes per minute, mean path-holding time and flow-rule updates per minute where supported. Model inference time and complete observation-to-verification time are measured separately. Paired differences will be calculated for matched workload seeds, with 95% confidence intervals and separate results for each trained initialization. The number of observations will be reported without treating successive windows as independent training runs (Agarwal et al., 2021).

Congestion events will follow the recorded operational thresholds: utilization of at least 85%, round-trip time of at least 100 ms, loss of at least 2%, or queue occupancy of at least 80% of known packet capacity. A qualifying breach requires three valid observations of the same metric spanning at least ten seconds, with no observation gap exceeding ten seconds. These values originate from the application's alert configuration and serve as operational criteria for the experiment. They will be fixed before test collection, with sensitivity analysis on validation data examining the effect of the selected boundary. Event onset is the first sample in a qualifying breach, while detection time is recorded separately. Separate incidents will require sustained recovery below the corresponding lower threshold: 70% utilization, 70 ms round-trip time, 1% loss or 40% queue occupancy.

Proactive intervention will be assessed against aligned no-intervention runs. Lead time equals reference congestion onset minus verified action time. A positive lead time identifies an early action relative to the reference event; successful avoidance additionally requires the intervention run to remain below the event criterion during the corresponding interval. Actions after onset count as reactive mitigation. Runs without a qualifying reference event will be reported without a lead-time value. This operational distinction follows the preventive and corrective traffic-engineering categories described by Farrel (2024).

A safeguard ablation will compare the same frozen policy with and without additional route-hold and change-rate restrictions inside the isolated experiment. Authorization, emergency stop and resource isolation remain active in both conditions. The comparison will report delivery, blocked proposals, actual route changes and recovery outcomes. Diagnostic outputs will expose policy probabilities, reward contributions and acceptance reasons. These provide evidence for inspecting decisions without implying that probability alone explains causation. Integrated autonomy will be evaluated only when observation compatibility, calibration, authorized actuation and recovery are available.

3.3 Software Development Methodology

The project uses Agile prototyping with time-boxed review cycles and a Kanban board. OOAD represents the application through objects with defined responsibilities,

including network inventory, telemetry records, routing models and execution services. This organization supports incremental integration and testing of individual components before the full workflow is evaluated.

3.3.1 Justification of the Methodology

Agile prototyping was selected because model behaviour, measurement quality and operator interaction require repeated evaluation. Time-boxed planning and review provide opportunities to assess a working increment, while visualizing work and limiting concurrent tasks help expose integration bottlenecks (Coleman et al., 2025; Schwaber & Sutherland, 2020). The approach is adapted to an individual project with supervisor reviews rather than presented as a complete Scrum team structure.

OOAD complements this process by allowing related data and behaviour to be developed within bounded components. Changing a telemetry representation requires review of its model-input contract, while an interface change should not independently alter execution authority. The analysis and design work in Chapter 4 will describe these responsibilities, and incremental tests will check the component relationships as the prototype evolves.

3.4 Methodology Diagram

Figure 3.1 presents the Agile prototyping lifecycle. It begins with requirements and prototype planning, proceeds through implementation and review, and concludes with final evaluation. Feedback from testing returns to the next refinement cycle, while model selection and final testing remain distinct activities.

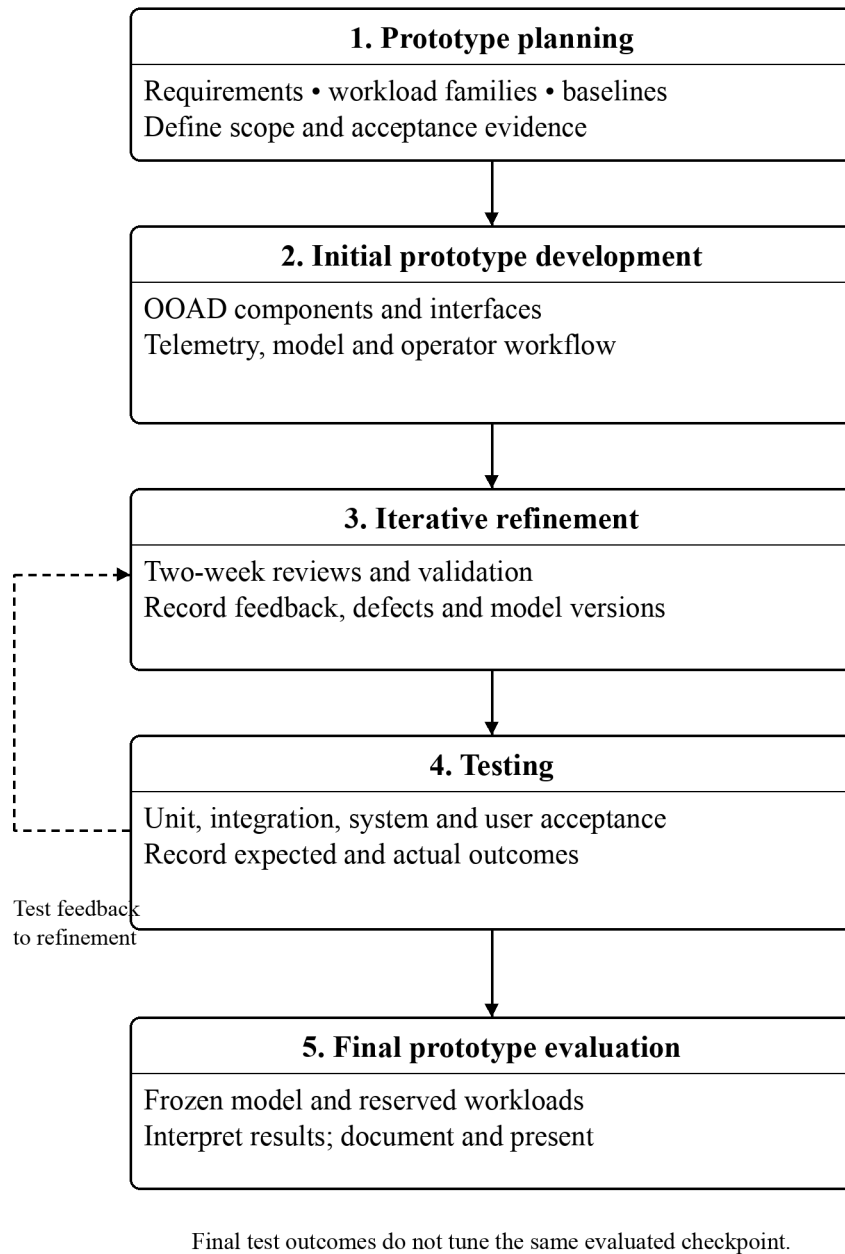


Figure 3.1. Agile prototyping lifecycle

Note. Original project-specific diagram informed by Coleman et al. (2025) and Schwaber and Sutherland (2020).

The feedback path permits changes to a prototype when a measurement, interface or integration check fails. A frozen release then enters reserved evaluation. Test outcomes inform the reported findings and subsequent work, while a further model change requires a separately declared experiment. Sections 3.4.1–3.4.5 explain the five stages.

3.4.1 Prototype Planning

Prototype planning identifies congestion-management requirements from the literature, existing application contracts and supervisor feedback. Priority functions are measured telemetry, candidate-route generation, reviewable execution and performance reporting. The plan specifies workload families, baselines and acceptance evidence before experiments begin. Requirements will be traced to the objectives and to the analysis and design developed in Chapter 4.

3.4.2 Initial Prototype Development

The initial implementation established controller connectivity, the bounded topology, traffic generation and the observation pipeline. The FastAPI application and React interface provide the operator workflow, while a separate PyTorch runtime handles the model. OOAD assigns responsibilities to inventory, telemetry, simulation and execution components. The prototype is extended incrementally so that a new observation or action can be checked before it is used in a learning experiment.

3.4.3 Iterative Refinement

Each two-week review cycle will examine the working increment, recorded defects and validation evidence. Supervisor feedback on demonstrations will be recorded and used to prioritize the next cycle. Model refinement will examine directional route choices, reward behaviour and sensitivity to workload, using training and validation partitions. A Kanban board will track planned, active, blocked and completed work; historical failed candidates will remain in the experiment record.

3.4.4 Testing

Unit tests check calculations, validation and individual service behaviour. Integration tests check contracts between telemetry, model diagnostics, intent processing and persistence. System tests exercise complete workflows, including approval, cancellation, recovery and reporting. Black-box operator scenarios complement white-box checks of internal logic. Planned user acceptance sessions will ask available authorized reviewers to inspect telemetry, assess a proposed route and interpret its outcome; participant counts, tasks and feedback will be reported only after those sessions occur.

Each test will identify its objective or requirement, input conditions, expected outcome, observed result and evidence reference. Negative cases include stale observations, insufficient permissions, expired approval and missing calibration. Recovery cases include worker interruption, duplicate messages, timed override expiry and emergency stop. Failed tests will retain their defect and retest records, supporting iterative correction rather than a verdict based on a single demonstration.

3.4.5 Final Prototype Evaluation

The final prototype will be assessed against the research objectives using the evaluation protocol in Section 3.2.4 and the recorded application tests. Evaluation will consider delivery, congestion, route stability and the completion of the operator workflow. The result will identify which acceptance criteria are met and which remain unresolved. Documentation and presentation preparation are targeted for November 2026.

3.5 System Development Tools and Techniques

The tools were selected for compatibility with the implemented prototype and the controlled evaluation. Table 3.1 identifies their roles. Dependency files, release identifiers and experiment configurations retain the information required to reproduce the application and its experiments.

FastAPI services expose authenticated workflows, and the React interface receives application state through APIs and WebSocket updates. Module-owned repositories and documented events organize data access. Git records implementation changes, while dependency files and container manifests retain the environment used for a release. A reproducible run requires matching model, observation contract, workload definition and runtime configuration.

Table 3.1: Development tools and their roles

Tool or environment	Role	Reason for selection
Mininet and Open vSwitch	Wired operator testbed.	Controlled topology and link settings using real network software.

Tool or environment	Role	Reason for selection
Ryu and OpenFlow 1.3	Operator control plane.	Explicit flow installation and switch communication.
Linux and FRRouting	Matched routing experiment.	Common forwarding conditions for OSPF and learning methods.
PyTorch and Gymnasium	Model and environment interface.	Inspectable tensors, policy updates and saved checkpoints.
FastAPI	Application services.	Typed request handling and bounded service responsibilities.
React, TypeScript and Three.js	Administrator and digital-twin views.	Interactive presentation of topology, telemetry and workflow state.
PostgreSQL, Neo4j and Redis	Persistence and coordination.	Relational records, topology queries and event delivery.
Docker and Git	Runtime packaging and version control.	Reproducible environments and traceable implementation changes.

3.5.1 Network Emulation and Control

Mininet and Open vSwitch provide the isolated operator topology. Ryu supports OpenFlow communication and approved flow changes, while Linux queue instrumentation supplements the available controller statistics. The separate FRRouting experiment makes nominal-cost OSPF behaviour observable under the same offered traffic used for policy comparison. The topology and configured limits bound the interpretation of the measurements.

3.5.2 Learning and Decision Inspection

PyTorch implements the categorical actor–critic model and automatic differentiation, while the environment interface validates observations and records transitions. The checkpoint contains the feature and action contract needed to reproduce inference. Policy probabilities, value estimates and reward components support decision

inspection. The learning method follows PPO, and its effect is evaluated against the baselines rather than assumed from the choice of algorithm (Schulman et al., 2017).

3.5.3 Operational Safeguards

The safeguard implementation assesses a bounded next-step queue envelope. In Equation 3.4, q_i is the observed queue in bytes, a_i^+ is an upper arrival-rate bound, s_i^- is a lower service-rate bound, Δt is the assessment horizon and e_i^+ is an error allowance. Valid bounds must correspond to the experiment and its observation interval.

$$q_i^+ = \max[0, q_i + (a_i^+ - s_i^-)\Delta t] + e_i^+$$

Equation 3.4: Conditional queue envelope

Equation 3.5 evaluates the change in a quadratic queue function against the configured drift budget B . Acceptance also requires current and bounded next-step queues to remain within their limits. Observation freshness, valid routes, minimum path-holding time and change-rate restrictions provide additional checks.

$$V(q) = \frac{1}{2}\sum_i q_i^2; V(q^+) - V(q) \leq B$$

Equation 3.5: Queue-function drift condition

The assessment is conditional on the supplied arrival, service and error bounds. It does not establish indefinite stability or independently authorize execution. This treatment follows the importance of model assumptions emphasized in safe-learning research (Brunke et al., 2022). Authorization, emergency stop, timed overrides and restoration remain application-level controls, and the planned ablation examines their relevant routing restrictions separately from the reward penalty.

3.6 System Deliverables

3.6.1 System Documentation

The documentation comprises the technical description, operator guidance, installation procedure, work plan and evaluation protocol. The experiment package will retain configuration, workload identifiers, model checkpoints, measurements and analysis outputs so that reported findings can be traced to their source.

3.6.2 Authentication Module

The authentication module provides password hashing, session management and role-based access. Organization and workspace membership restrict the resources available to a user. Monitoring, proposing changes and approving execution remain separately controlled functions.

3.6.3 Telemetry Module

The telemetry module stores scoped measurements with source, units and timestamps, supports historical queries and supplies current observations to the interface. The associated alert workflow records sustained breaches and recovery. These functions provide the data needed for congestion characterization and evaluation.

3.6.4 Cognitive Routing and Safeguard Module

The routing deliverable combines the versioned PPO model, its observation contract and the guarded decision workflow. The actor proposes a path, the safeguard layer records acceptance or rejection reasons, and authorized execution produces readback evidence. Outstanding compatibility and calibrated-control tests remain part of prototype validation.

3.6.5 Administrator Dashboard and Reports

The dashboard presents topology and digital-twin views, telemetry, simulation comparisons, intent review, diagnostics and audit records. It supports approved manual actions, timed overrides and downloadable reports. The final evaluation will document how these functions support operator inspection of the routing experiment.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter translates the congestion-management problem into the requirements and design of the Neuro-Adaptive Network Flow Orchestrator (NANFO). It explains the system boundary, the information needed to make a routing decision, the responsibilities of the application components, and the controls governing changes to the emulated network. The analysis follows the Object-Oriented Analysis and Design approach established in Chapter 3, while the learning and network-evaluation components retain their experimental character.

The central design problem is not simply to select a less congested path. An operationally useful system must establish which network produced an observation, determine whether that observation is sufficiently current, evaluate a candidate action, check the authority to apply it, and retain evidence of the resulting configuration. Sections 4.3 and 4.4 present the corresponding use-case, activity, state, system-architecture, software-class, logical-data and sequence views, together with the model pipeline, operator wireframe and deployment boundaries. The diagrams model the requirements and documented implementation contracts (Bhudia, 2026). UML views follow Object Management Group (2017), while the specialised block views state their own connector meanings.

4.2 System Requirements

The requirements were derived from the research objectives, the limitations identified in Chapter 2, and the implemented application contracts (Bhudia, 2026). They distinguish the behaviour required of the complete system from the evidence currently available to verify it. This distinction is important because a requirement for safeguarded autonomous routing is not satisfied merely by implementing a mode selector or demonstrating a trained model in a separate experiment.

ISO/IEC/IEEE 29148:2018 provides a framework for requirements engineering and its associated information items (International Organization for Standardization et al., 2018). Accordingly, the catalogue below assigns identifiers to requirements, states the required behaviour, and identifies how that behaviour can be examined. The identifiers provide a common reference for implementation and testing in Chapter 5.

They are traceability labels for this documentation, rather than a claim that the original development backlog used the same labels.

4.2.1 Functional Requirements

The principal user is an authorized network operator. Depending on current permissions and organization membership, the operator may inspect observations, configure resources, review an intent, approve execution, request restoration or retrieve reports. A read-only operator can inspect permitted resources but cannot acquire write authority through a client-side control. Model training is a separate research operation, and the emulated network acts as an external source of observations and a target of authorized changes.

Table 4.1 presents the core functional requirements. The verification column identifies the necessary evidence; whether that evidence has been obtained is evaluated in Chapter 5.

Table 4.1: Functional requirements

ID	Requirement	Verification method
FR-01	The system shall authenticate an existing active user before granting access to protected functions.	Valid/invalid session tests and authenticated workflow execution.
FR-02	The system shall restrict resource access according to current permissions and organization/workspace membership.	Cross-tenant, read-only, disabled-user and membership-revocation cases.
FR-03	The system shall maintain network and device identities that can be mapped to the resources observed in the authorized emulator.	Inventory/binding inspection and topology comparison.

ID	Requirement	Verification method
FR-04	The system shall store valid measured telemetry with its resource identity, observation time, metric, value, unit and provenance.	Source-to-record comparison, invalid-input and duplicate-replay tests.
FR-05	The system shall record congestion-related incidents when the configured sustained measurement conditions are met.	Timed breach/recovery tests and incident-history inspection.
FR-06	The system shall evaluate an explicitly configured network scenario and retain its modelled outputs and lifecycle state.	Deterministic replay, accounting, pause/resume and comparison tests.
FR-07	The system shall retain a proposed network change and the evidence used to assess it before execution.	Intent, plan, validation and evidence-identity inspection.
FR-08	The system shall apply a supported manual lab action only after the required approval and current execution checks pass.	Authenticated reroute, multipath, shaping/policing and rejected-dispatch tests.
FR-09	The system shall reconcile cancellation or timed-override expiry with verified restoration or an explicit unresolved state.	Expiry, interruption, revocation, emergency-stop and rollback cases.

ID	Requirement	Verification method
FR-10	The system shall evaluate an approved frozen checkpoint against an explicitly selected compatible observation for diagnostics.	Checkpoint/input verification, repeated inference and confinement tests.
FR-11	The system shall generate downloadable reports from authorized source records and retain their integrity metadata.	Rendering, parsing, owner-access and size/hash verification.
FR-12	The system shall permit an autonomous change only when model, observation, calibration, policy and execution-authorization requirements are satisfied together.	Integrated closed-loop acceptance and missing-prerequisite rejection tests.

Authentication does not include a public registration service. The deployment process provisions an initial administrator, after which access follows the application's existing identity and membership mechanisms. This avoids introducing an account-creation function that is not part of the implemented interface.

Telemetry and incident management are related but distinct functions. A telemetry record describes an observation; an incident describes a sustained condition inferred from a sequence of eligible observations. Acknowledging an incident records operator awareness, while resolving it changes its lifecycle state. The system must preserve the underlying observations so that an incident can be examined without relying solely on a coloured dashboard indicator.

The manual and autonomous execution requirements also differ. A supported manual command can be authorized for a bounded laboratory action after its own checks succeed. Autonomous execution additionally requires a compatible qualified model

and the calibrated decision conditions of FR-12. The implementation must communicate this difference rather than imply that every successful manual reroute certifies autonomous operation.

4.2.2 Non-Functional Requirements

The relevant quality concerns are access control, measurement integrity, recovery, reproducibility, operator interaction and controlled resource use. ISO/IEC 25010:2023 provides a product-quality model for specifying and evaluating such properties (International Organization for Standardization & International Electrotechnical Commission, 2023). The requirements in Table 4.2 apply these concerns to NANFO without treating the standard as a certification of the application.

Table 4.2: Non-functional requirements

ID	Requirement	Acceptance basis
NFR-01	Authorization shall be enforced by the server for each protected operation.	Current identity and membership checks; a browser control or token role snapshot cannot independently grant access.
NFR-02	Reprocessing the same accepted observation or completed lifecycle event shall not create a duplicate logical record.	Stable event/observation identities, uniqueness constraints and replay cases.
NFR-03	Stale observations shall not be presented as evidence of a current healthy state or used for a new permitted decision.	Source-specific freshness checks and explicit stale/unavailable presentation.
NFR-04	An evaluated model shall remain reproducible from its checkpoint, feature contract and recorded input.	Identical action/probability/value replay under the matching frozen runtime.

ID	Requirement	Acceptance basis
NFR-05	Long-running work shall expose a persisted lifecycle outcome rather than depend on an uninterrupted browser session.	Independent worker execution, interruption and later-status queries.
NFR-06	The interface shall distinguish observations, predictions, proposals and verified actions.	Contract inspection, labelled views and operator workflow tests.
NFR-07	Missing or incompatible autonomous-control evidence shall prevent autonomous dispatch.	Negative tests for missing providers, stale evidence, invalid bounds and absent authority.
NFR-08	The scenario evaluator shall account for offered traffic as delivered, dropped, queued or in flight.	Numerical accounting and deterministic continuation tests.
NFR-09	A claim of routing improvement shall satisfy the evaluation criterion declared for that experiment.	In the retained benchmark, positive lower goodput-difference confidence bound and negative upper RTT-difference bound against nominal-cost OSPF.
NFR-10	Components shall access another module's data through its public contract rather than direct cross-module database queries.	Service/repository inspection and integration-contract review.

ID	Requirement	Acceptance basis
NFR-11	Deployment shall support an explicit, inspectable recovery procedure for persistent application state.	Backup/restore implementation checks and a separate fresh-restore acceptance campaign.

Freshness is evaluated at several points because an observation can become stale between collection, display and execution. The emulation reader accepts observations within its configured age limit, whose default is thirty seconds. The measured-alert detector has its own thirty-second eligibility limit. The digital-twin display expires congestion indications after sixty seconds, using a five-second presentation heartbeat. These settings serve different purposes and should not be interpreted as one universal network sampling interval.

Performance requirements are similarly tied to the relevant activity. Neural inference time, application response time, control/readback time and packet round-trip time measure different processes. The retained experiment specifies a comparative delivery criterion, while the frontend has a separate compressed-JavaScript budget of 410.16 KiB. Neither establishes a production latency guarantee. A requirement such as “all changes complete within milliseconds” would be unsupported by the measured control-loop timings.

Reliability requires an explicit distinction between retry tolerance and durable recovery. NFR-02 concerns duplicate logical effects under replay. It does not mean every source observation survives a collector crash. Likewise, NFR-11 requires a recovery procedure, but its live acceptance remains separate from the existence of backup code. These distinctions allow Chapter 5 to assess the weakest unverified link in an otherwise functioning workflow.

4.2.3 Data Requirements

NANFO handles three principal categories of data: application records, measured network observations and experimental artefacts. Application records include identities, memberships, inventory, intents, simulations, incidents and reports. Observations describe the state measured at a particular resource and time. Experimental artefacts include workload definitions, checkpoint manifests, raw

response traces and comparison results. Their provenance differs, and the storage design must preserve that distinction.

DR-01 — Resource and observation identity. Every accepted measurement shall identify its network, device and workspace, together with its metric, unit, source and observation timestamp. Port, peer and run identifiers shall be retained when they distinguish measurement series. Two observations from different ports or runs must not be merged solely because they share a metric name.

DR-02 — Measurement semantics. Missing measurements shall remain unavailable. A measured zero may be stored only when the instrument reports it. Round-trip time shall remain distinguishable from one-way delay, probe loss from application-delivery loss, and port throughput from useful received payload. A modelled result shall carry its model-derived provenance.

DR-03 — Validation and temporal order. Input records shall satisfy the versioned schema, finite-value checks and permitted time/identity relationships. Invalid durations, contradictory packet counts, duplicate resource identities and stale observations shall be rejected or explicitly excluded. Counter resets shall invalidate a rate calculation that requires a previous monotonic sample.

DR-04 — Experimental partitioning. Training, validation and final-test workloads shall retain separate identifiers. Model selection shall use validation evidence, and final-test observations shall not be reused to train the evaluated checkpoint. Related transitions must remain associated with their episode and workload seed rather than be treated as independent records drawn at random.

DR-05 — Artefact provenance. Checkpoints, effective configuration and evidence packages shall retain version and integrity identifiers. A hash establishes correspondence with recorded bytes; it does not independently prove measurement truth, authorize execution or replace the missing artefact itself.

DR-06 — Retention and access. Sensitive identity data and control authority shall remain separate from the producer's observation files. Reports and experiment records shall be retained under their documented access controls. Deletion shall respect the owning module and any evidence dependencies. A complete telemetry-deletion policy

requires an evidence-pinning contract; where that contract is absent, retention limits must be reported rather than solved through unverified deletion.

The emulation data contains generated network addresses and synthetic workload identifiers rather than institutional user traffic. This reduces the need to collect personal packet contents for the research question. It does not eliminate the need to protect credentials, sessions and operator records in the application. Password hashes and authentication tokens have no legitimate role in telemetry events, exported reports or screenshots.

4.2.4 Interface and Integration Requirements

IR-01 — Application interface. JSON REST responses shall use the documented envelope containing success, data, meta and errors. Binary report downloads and bodyless responses shall follow their specific contracts. Resource identifiers and execution-mode metadata shall remain available where required by those contracts. Validation errors must not expose internal stack traces or credentials.

IR-02 — Realtime interface. WebSocket delivery shall require an authenticated session and authorized resource scope. Subscription acknowledgement shall establish readiness for the selected stream. Reconnection shall trigger bounded reconciliation with authoritative REST state so that a restored socket is not mistaken for a complete event history.

IR-03 — Observation handoff. The isolated lab shall publish the versioned snapshot contract, and the backend shall verify the trusted mapping between emulator identities and application resources. The producer shall not be permitted to choose an arbitrary tenant or rewrite its own authorization binding.

IR-04 — Model interface. The learning runtime shall receive the feature representation expected by its checkpoint and return the declared action/value outputs. Diagnostic callers shall select an allowed checkpoint and historical input through the registry contract rather than supply an arbitrary executable or filesystem path.

IR-05 — Execution handoff. The execution worker shall send a bounded command through the provisioned mailbox and reconcile the matching result. The command, run, binding and execution identities shall remain consistent during retry and cancellation. Only the authorized worker shall write command files.

The controller-to-switch protocol in the operator laboratory is OpenFlow 1.3. The backend does not act as a second independent switch controller. It receives observations through the file contract and requests permitted actions through the execution boundary. This division keeps controller-specific mechanisms in the laboratory component while the application owns identity, workflow state and auditability.

4.2.5 Requirements Traceability

The five research objectives retain the order established in Chapter 1: characterize congestion and its challenges; compare existing solutions; identify their limitations; develop the integrated prototype; and evaluate its performance. The first three establish the rationale for requirements, while the fourth and fifth require implementation and test evidence.

Table 4.3 connects these objectives to requirement groups and the principal Chapter 5 evidence. A documented relationship is not itself a passing verdict; the test outcomes determine whether a requirement has been met within the stated conditions.

Table 4.3: Objective-to-requirement traceability

Objective	Relevant requirements	Principal implementation/evaluation evidence
i. Congestion indicators and challenges	FR-03–FR-05; DR-01–DR-03; NFR-03	Telemetry pipeline, counter checks, historical series and measured alerts; TC-03–TC-06.
ii. Existing approaches and contributions	NFR-06; IR-01–IR-02	Chapter 2 comparison translated into evidence-linked operator views and controlled workflows.

Objective	Relevant requirements	Principal implementation/evaluation evidence
iii. Timing, stability and reproducibility gaps	FR-09–FR-12; NFR-02, NFR-04, NFR-07, NFR-09; DR-04–DR-05	Replay, restoration, model qualification and bounded interpretation of the held-out comparison; TC-09–TC-16.
iv. Integrated prototype	FR-01–FR-11; NFR-01, NFR-05, NFR-08, NFR-10; IR-01–IR-05	Authentication, inventory, telemetry, simulation, manual execution, diagnostics and reporting; TC-01–TC-13.
v. Comparative validation	FR-12; NFR-09, NFR-11; DR-04–DR-06	Matched benchmark, regression/acceptance records and unresolved integrated-autonomy and recovery cases; TC-14–TC-19.

4.3 System Analysis Diagrams

4.3.1 System Boundary and Actors

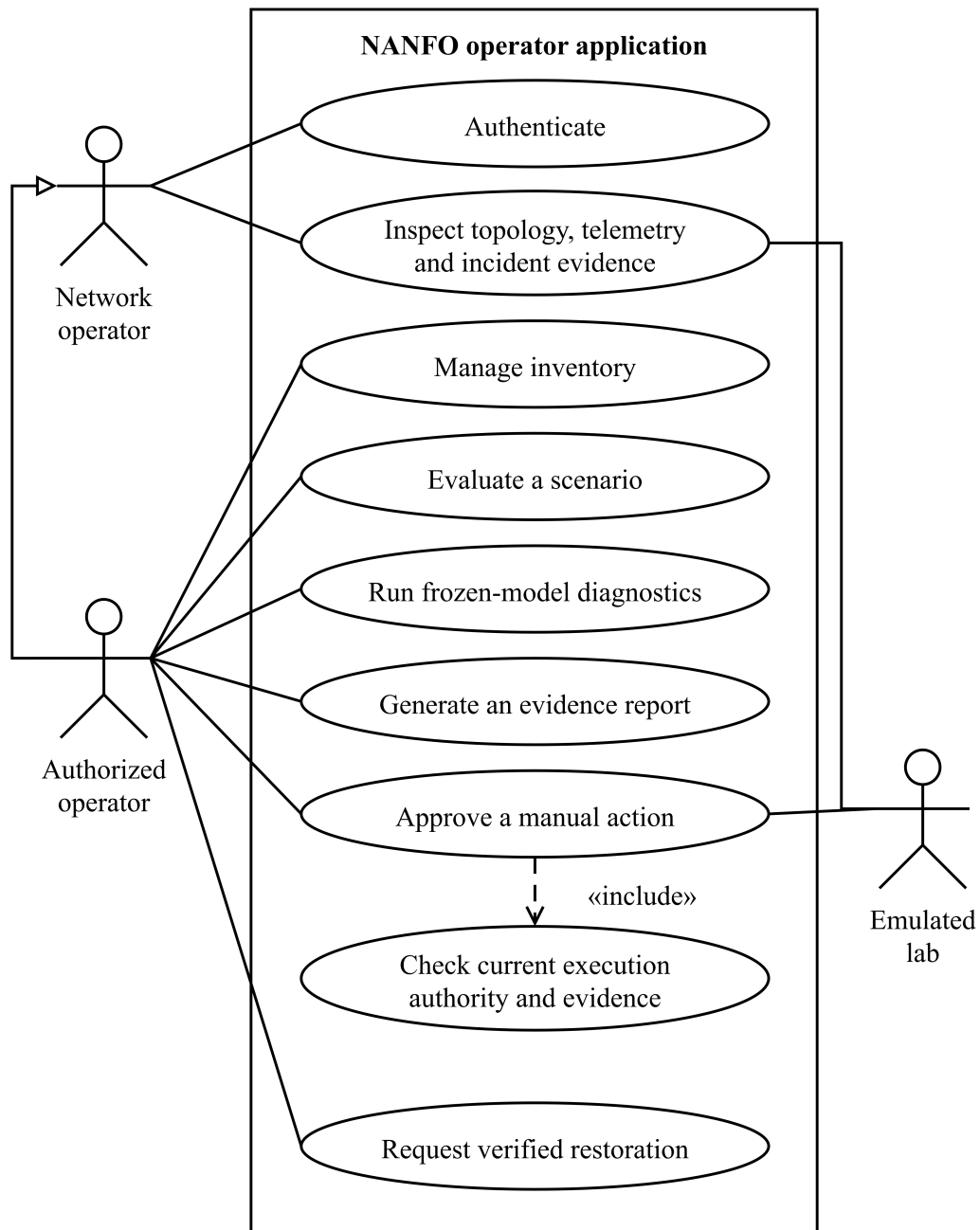
The system boundary includes the operator application, its owning services and repositories, and the workers that process simulations, execution, recovery, alerts and reports. The emulated network lies outside this application boundary. It supplies observations and receives commands through explicitly configured interfaces. The separately packaged learning runtime also has a defined input/output boundary, even when it is launched from the same host.

The network administrator manages the authorized workspace, reviews network condition and controls permitted changes. The read-only operator inspects permitted information without modifying policy. A research operator conducts bounded training and evaluation campaigns, using the same project artefacts but a distinct experimental workflow. These descriptions identify responsibilities; they do not create new

application role names or imply that every authenticated user can conduct a training run.

A supervisor or examiner is a recipient of evidence rather than an internal application actor unless an appropriate account has actually been provisioned. Databases, neural-network classes and internal workers are components of the implementation, not external human actors. The use-case view therefore models external participation without representing implementation dependencies as user requirements.

Figure 4.1 identifies the external participants and the principal operator tasks for FR-01–FR-11. A specialized authorized operator inherits the network operator’s inspection and authentication tasks. The solid hollow-triangle generalization points to that general actor; the dashed include arrow identifies mandatory execution checks (Object Management Group, 2017).



Actor specialization expresses responsibilities; the server checks each permission. Protected tasks require an active session and the current resource scope.

Figure 4.1. Application actors and use cases

The generic network-operator actor expresses observation responsibilities, while the specialized actor adds permission-scoped changes, diagnostics and report generation. This is a responsibility model rather than an additional application role. The lab exchanges measured observations and bounded command results. Authentication remains a precondition of protected tasks, and a requested restoration is complete only after the recovery evidence in Figure 4.4 is verified.

4.3.2 Observation-to-Assessment Workflow

The workflow begins when the lab collects switch counters, queue information and endpoint probe results. The application first determines whether the snapshot belongs to the expected topology and authorized run. It then maps observed resources to the inventory, validates timestamps and derives measurements whose calculations require consecutive samples. A throughput value, for example, requires a valid byte-counter difference and elapsed counter duration.

Validated observations enter the telemetry event path and are persisted under stable identities. Historical queries and realtime delivery expose the same measurement meaning to the operator. The alert service examines eligible persisted observations against its sustained threshold rules. This ordering matters: an incident should be traceable to a stored observation rather than depend on an unverified value supplied by a replayed notification.

An assessment can then use the current observations or an explicitly selected historical input. The distinction is maintained in both the model interface and the display. A successful replay against a historical checkpoint answers whether the recorded decision can be reproduced; it does not establish that the same action is suitable for the network's present state.

Figure 4.2 traces one observation batch from the laboratory handoff to persistent measurements and eligible incident assessment. It addresses FR-03–FR-05, DR-01–DR-03 and NFR-02–NFR-03. Rounded actions, decision diamonds, guarded solid control-flow arrows and initial/final nodes follow UML activity notation (Object Management Group, 2017).

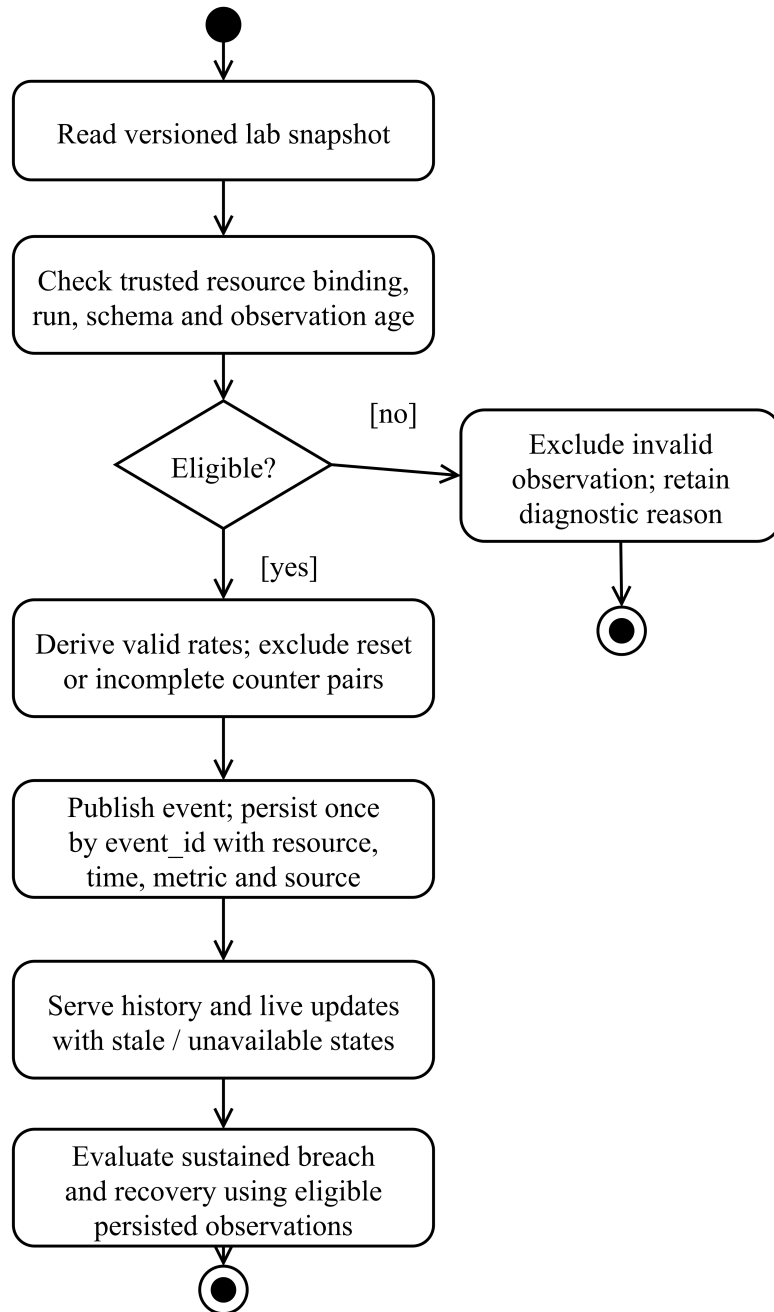


Figure 4.2. Observation validation and assessment activity

A failed binding, time or measurement check prevents the affected observation from entering assessment. Invalid counter-derived rates are excluded rather than converted to zero. Persistence uses stable event identities, so replay does not create a second logical observation. Incident evaluation depends on eligible stored evidence; a display refresh or a repeated notification does not establish a new sustained congestion condition.

4.3.3 Proposal, Approval and Execution Workflow

An intent expresses a desired network change. The application resolves it into a supported plan and records the plan identity, network scope and relevant validation evidence. Where configured simulation evidence is supplied, it is bound to the candidate action and checked before dispatch. The finite-buffer simulation provides modelled evidence; the current manual-lab workflow does not treat it as an unconditional certificate of physical safety.

Approval is a server-checked action. Before execution, the system re-examines the current actor, membership, binding, observation age and permitted action. The worker then communicates with the lab using the recorded command identity. A successful submission means the request has entered the workflow; completion requires the corresponding result and verification evidence.

Configuration readback and packet behaviour answer different questions. Readback establishes that expected rules are present, and reachability establishes that selected probes receive replies. Measured application throughput requires an additional traffic workload. Similarly, the selected-probe capture records observed packet paths during its capture interval; it is more specific than a planned route but does not describe every application flow on the network.

Figure 4.3 follows a supported manual action from its persisted plan to a verified outcome, addressing FR-07–FR-09 and IR-05. The activity separates approval, current admission checks, dispatch and result reconciliation. Simulation evidence is checked where required by the configured workflow; it is not treated as a universal certificate.

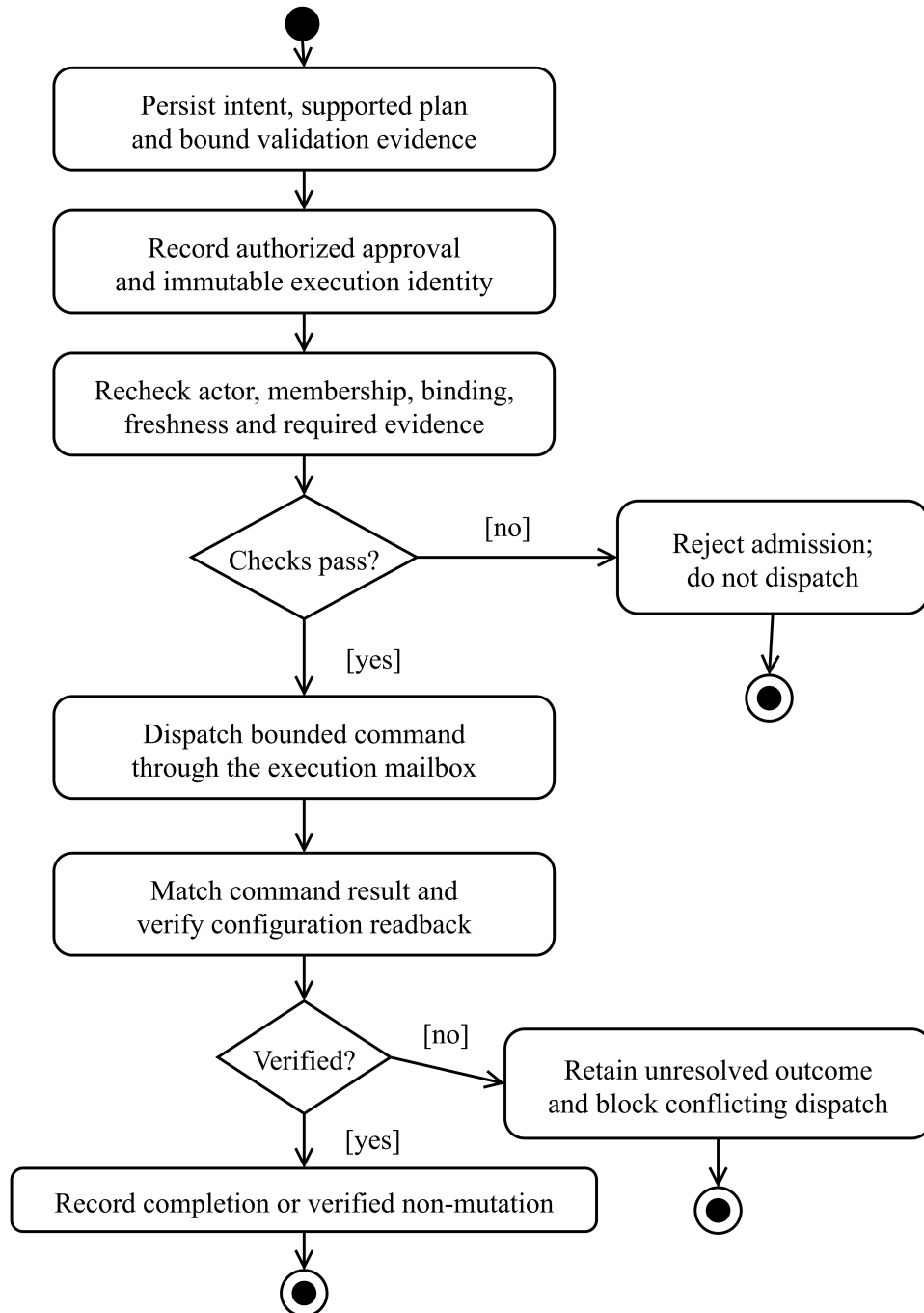


Figure 4.3. Approval and verified manual execution

An admission failure ends the attempt before dispatch. After a command may have reached the lab, an absent or inconsistent result retains an unresolved record and laboratory exclusion. A verified outcome may establish completion or confirmed non-mutation. Configuration readback and rollback evidence determine that distinction; application goodput requires separate measurement. Figure 4.4 develops the recovery path for a timed override.

4.3.4 Cancellation, Override and Failure Behaviour

A timed override belongs to a particular verified manual execution. Its expiry is enforced by a server-side worker rather than a browser countdown. Cancellation, loss of authority or emergency stop initiates reconciliation of that execution. The recovery process must establish that its owned flows were removed or that no mutation occurred before the network is released for another permitted change.

An ambiguous outcome is retained as unresolved. This is necessary when a command may have reached the lab but a response was lost, or when compensation cannot yet be verified. Automatically declaring success in such a case would improve the appearance of availability while weakening the meaning of the execution record. Retry therefore preserves identity and seeks evidence of the existing operation rather than creating a different operation with a new identifier.

Restoration does not automatically authorize a return to autonomous operation. The application rechecks the control revision, emergency-stop state, relevant authority and provider readiness. A failed return leaves the network in an explicitly permitted state. The design thus separates recovery from the decision to resume learning-driven control.

Figure 4.4 models the persisted timed-override lifecycle for FR-09 and NFR-05. State names correspond to the Autonomy records. Solid arrows describe transitions; bracketed guards state the evidence required to advance. The diagram separates physical restoration from permission to return to the requested operating mode.

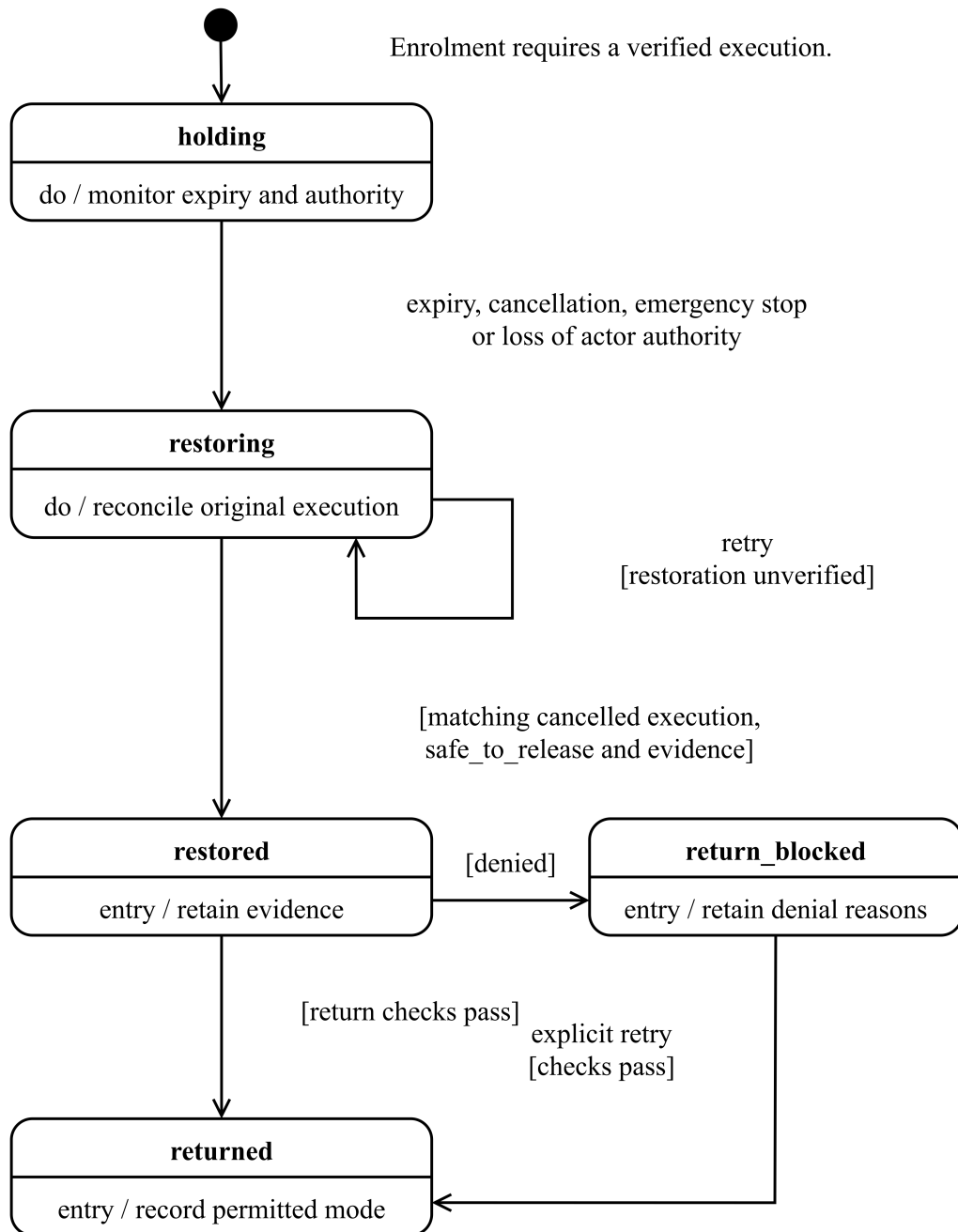


Figure 4.4. Timed-override restoration and guarded return

The restoring state is retained while restoration evidence is uncertain, so a lost response cannot release the laboratory prematurely. Only a matching cancelled

execution with safe-to-release evidence permits restored. Return checks then examine the current control revision, stop state, authority and provider readiness. A blocked return does not undo verified restoration, and returned denotes the requested mode transition rather than proof of autonomous-routing acceptance.

4.3.5 Information and State Analysis

The application distinguishes five states of knowledge: unavailable information, a measured observation, a modelled prediction, an intended configuration and a verified outcome. Collapsing these categories would make the interface difficult to interpret and the evaluation difficult to reproduce. For example, a zero-valued prediction must not stand in for a missing observation, and an accepted intent must not be displayed as a measured reroute.

Time and identity connect these states. The same numerical value observed on another port, under another workload or before a topology change has a different meaning. The data model consequently preserves resource scope and provenance rather than maintaining a single unqualified “network health” value. This principle connects the measurement challenges in Chapter 2 with the concrete design of the records and interfaces.

4.4 System Design Diagrams

4.4.1 System Architecture and Class Design

The backend is organized as a FastAPI modular monolith. Routes validate and expose the interface; services implement workflow and authorization rules; repositories persist data owned by their module. Modules share an application deployment and PostgreSQL infrastructure, but ownership boundaries constrain how data is accessed. Documented public services and domain events support integration without direct cross-module table queries.

Identity owns authentication and audit records. Organization owns memberships and workspaces. Network owns inventory and campus-model assets. Telemetry owns measurement persistence and history. Simulation owns numerical scenarios and lifecycle state. Intent owns plans, execution jobs and execution-event publication. Autonomy owns control configuration, decision history and timed overrides. Alert

and Report own their respective records and background work. Plugin functionality is limited to a metadata registry and is not an executable extension platform.

Independent workers separate long-running operations from request handling. This permits the interface to retrieve the current lifecycle state after a connection is interrupted and allows recovery work to continue without an open browser. A Redis lease restricts realtime API ownership to the supported single-instance arrangement. The design therefore supports modular development and controlled background processing without claiming a horizontally scaled application architecture.

Figure 4.5 presents NANFO’s system architecture for FR-03–FR-12, NFR-05, NFR-10 and IR-01–IR-05. Dashed dependencies point toward required contracts. Module ownership, background processing, laboratory integration and persistence remain explicit.

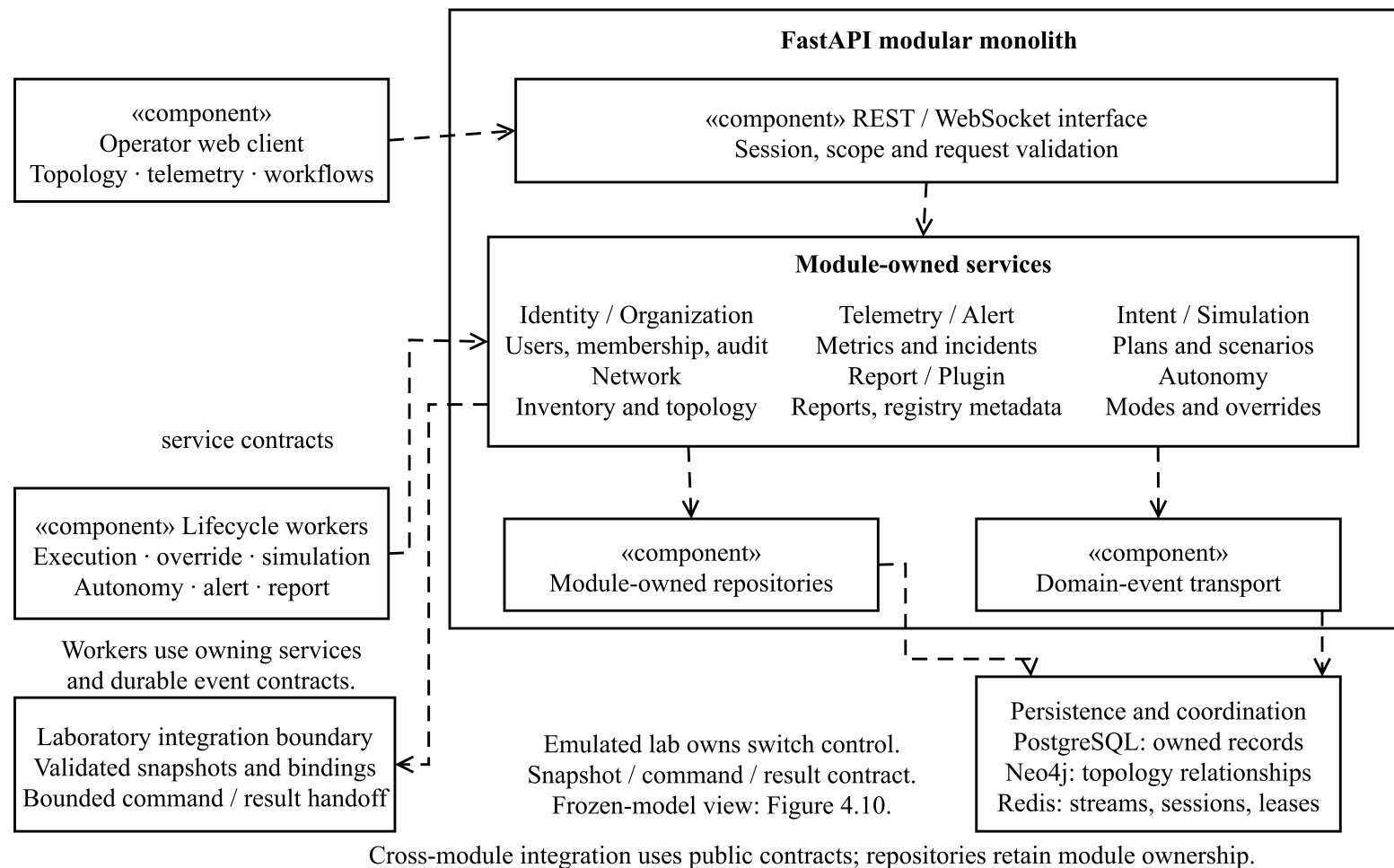
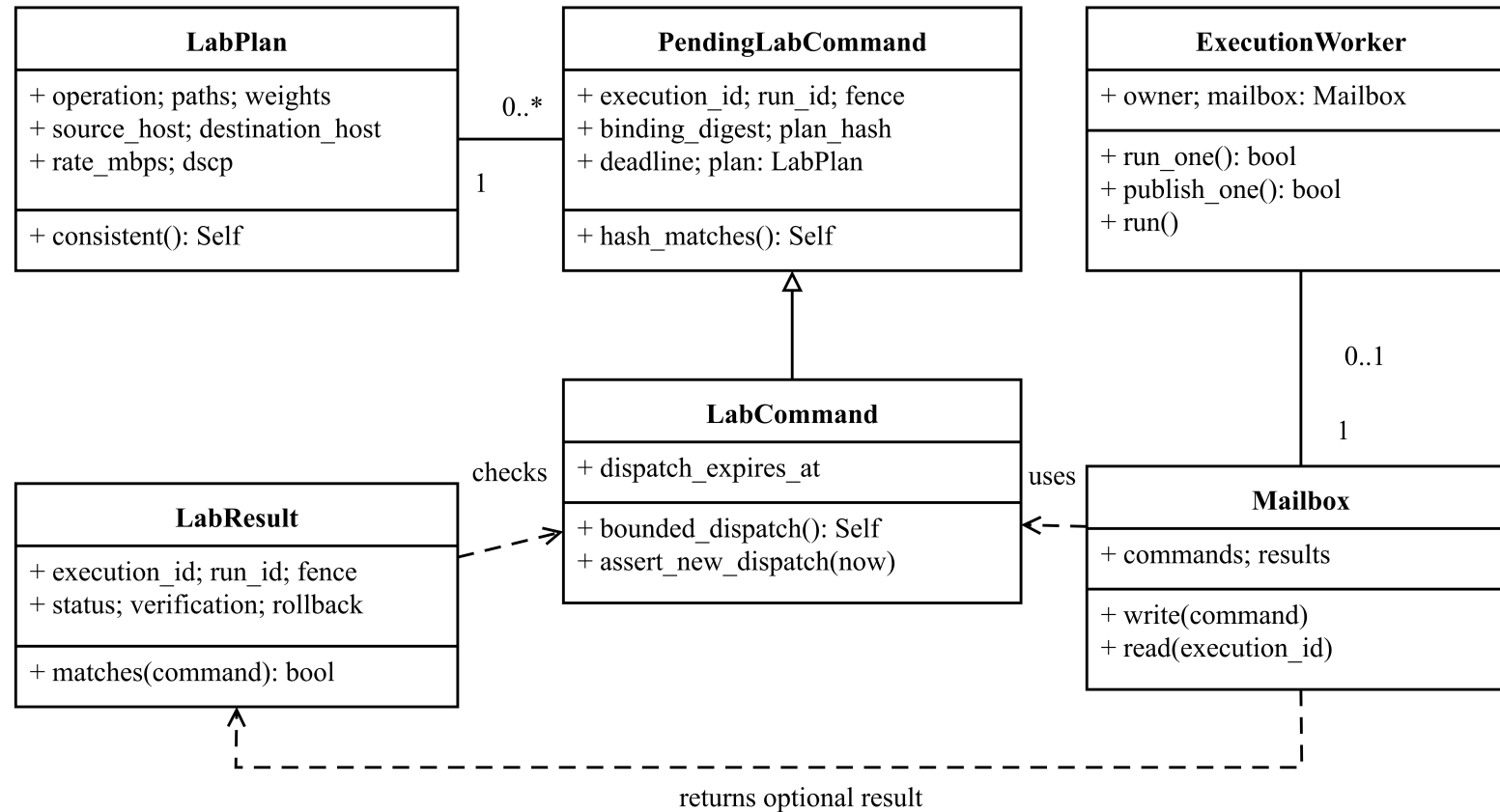


Figure 4.5. System architecture and module integration

Routes call owning services, while repositories retain module ownership. Workers continue independently of the browser. The laboratory boundary carries observations and bounded commands; PostgreSQL, Neo4j and Redis retain distinct storage and coordination responsibilities.

Figure 4.6 complements the architecture with a focused UML software-class view of the execution boundary (FR-07–FR-09 and IR-05). Class compartments show selected actual attributes and operations. A solid hollow-triangle arrow denotes inheritance, solid associations denote held object references, and dashed arrows denote use or type dependencies (Object Management Group, 2017).



+ public member; signatures abbreviated. Inheritance and relationships are code-derived; selected members are shown.

Figure 4.6. Command execution classes and responsibilities

LabCommand adds expiring dispatch authority. LabResult verifies execution, run, binding, plan and fence identities. ExecutionWorker uses its mailbox for bounded commands and matching results; Figures 4.7–4.8 cover persisted records.

4.4.2 Logical Data Design

The logical schema separates transactional application records from topology relationships and experimental artefacts. PostgreSQL stores module-owned records. Neo4j represents the topology graph used for relationship queries and views. Redis supports event streams, session state and short-lived coordination. Checkpoints and raw experiment packages remain versioned files whose identity is recorded separately from relational workflow state.

Table 4.4 summarizes the principal entities and relationships. It describes the logical design; exact DBMS types and indexes are implementation details discussed only where they affect correctness or evaluation.

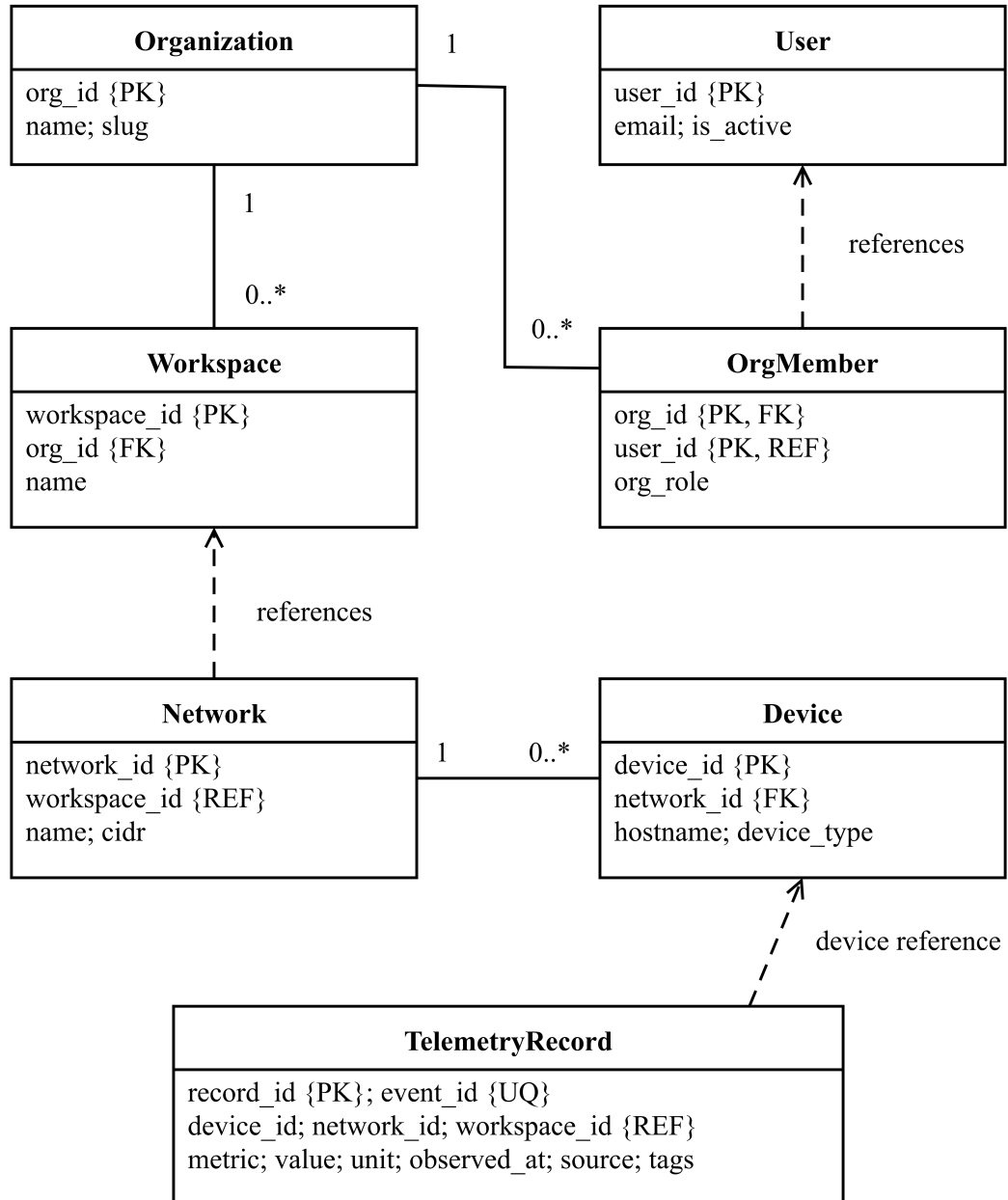
Table 4.4: Logical data entities and relationships

Entity/group	Principal information	Relationship and ownership
User, role and user-role assignment	Active identity, password hash and assigned role identifiers.	Identity-owned user-to-role association; sessions are managed separately in Redis.
Organization, workspace and membership	Organization identity, workspace identity, member and organization role.	Organization owns enforced internal relationships; the user identifier is a logical Identity reference.
Network and device	Managed network, device identity, address, type and status.	Network owns both entities and their one-to-many relationship.
Campus model and device groups	Spatial assets, building information and group membership.	Network-owned assets and resource references supporting the twin.
Telemetry record	Event identity, resource scope, metric, value, unit, observation time and source tags.	Telemetry-owned append-oriented observations with logical device/network/workspace references.

Entity/group	Principal information	Relationship and ownership
Intent and execution	Requested change, persisted plan, approval, execution identity and outcome.	Intent owns execution relationships and its publication outbox.
Simulation and checkpoint state	Scenario input, parent/branch identity, lifecycle, accounting and modelled output.	Simulation-owned scenario relationships and event publication.
Autonomy control, decision and override	Mode/revision, qualification reasons and exact manual-execution reference.	Autonomy owns control state; execution recovery uses the Intent service contract.
Alert, detector and history	Rule identity, observations, sustained state and incident lifecycle.	Alert-owned incident and history relationships.
Report and artifact metadata	Request scope, source snapshot, rendering state, byte count and digest.	Report-owned records with protected external artifact storage.
Audit log	Actor, event, resource, correlation and timestamp.	Identity-owned immutable history with event uniqueness.

The use of logical references across modules is deliberate. It avoids coupling every service to another module's physical table structure, but it transfers responsibility for reference validation to the owning service contract. Within a module, foreign keys enforce relationships such as a device belonging to a network or an execution belonging to an intent. The design must not portray all cross-module references as database-enforced foreign keys when they are not implemented that way.

Figure 4.7 gives a UML class-style logical schema for identity, resource scope and measurements, addressing FR-02–FR-04 and DR-01. Class compartments list selected stored attributes. Solid associations carry multiplicities; dashed dependencies identify logical references across ownership boundaries. PK, FK and REF are schema annotations defined in the figure.



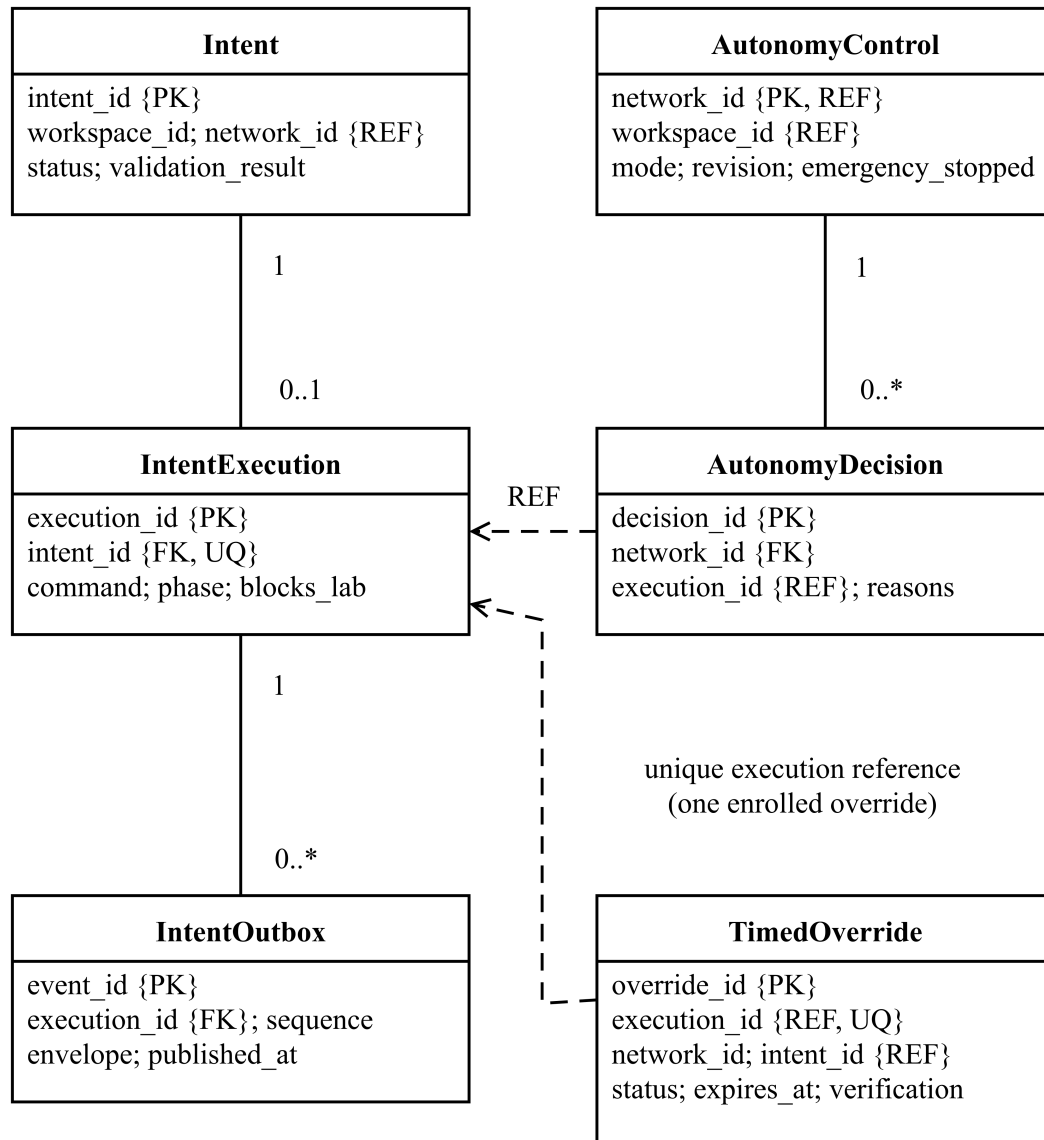
PK = primary key; FK = within-module foreign key; REF = logical reference.
UQ = unique. 1 = exactly one; 0..* = zero or more. Selected attributes shown.

Figure 4.7. Logical identity, scope and observation schema

Organization and Network enforce their own parent–child relationships. A network’s workspace identifier and a telemetry record’s resource identifiers are logical references validated through service contracts. They are not cross-module SQL foreign keys. The composite membership key prevents duplicate user membership within an organization, while the unique telemetry event identifier prevents repeated delivery from becoming a second observation.

Event identities provide a second form of integrity. A unique telemetry event identifier prevents a retry from becoming an additional observation, while lifecycle and audit identities allow consumers to recognize previously processed events. Immutability is particularly important for audit history, configuration revisions and historical evaluation evidence: a later successful attempt should be added to the record rather than used to rewrite an earlier failure.

Figure 4.8 expands the logical schema around FR-07–FR-09 and FR-12. It shows the cardinality between an intent and its execution, the owned publication outbox and the Autonomy records that refer to execution evidence. The same key and relationship conventions used in Figure 4.7 apply.



PK = primary key; FK = within-module foreign key; REF = logical reference.
 UQ = unique. Multiplicities label solid associations; dashed arrows denote reference dependencies. Additional fields and relationships are omitted.

Figure 4.8. Logical execution, publication and control schema

The unique intent identifier on IntentExecution limits an intent to zero or one execution record. Each execution may produce several outbox events, identified independently and ordered within that execution. Autonomy retains its own control

and decision history; execution and override references cross the service boundary without creating SQL foreign keys to Intent. This preserves recovery identity while avoiding cross-module table access. The execution-and-sequence pair is unique within the publication outbox, preventing duplicate logical publication positions.

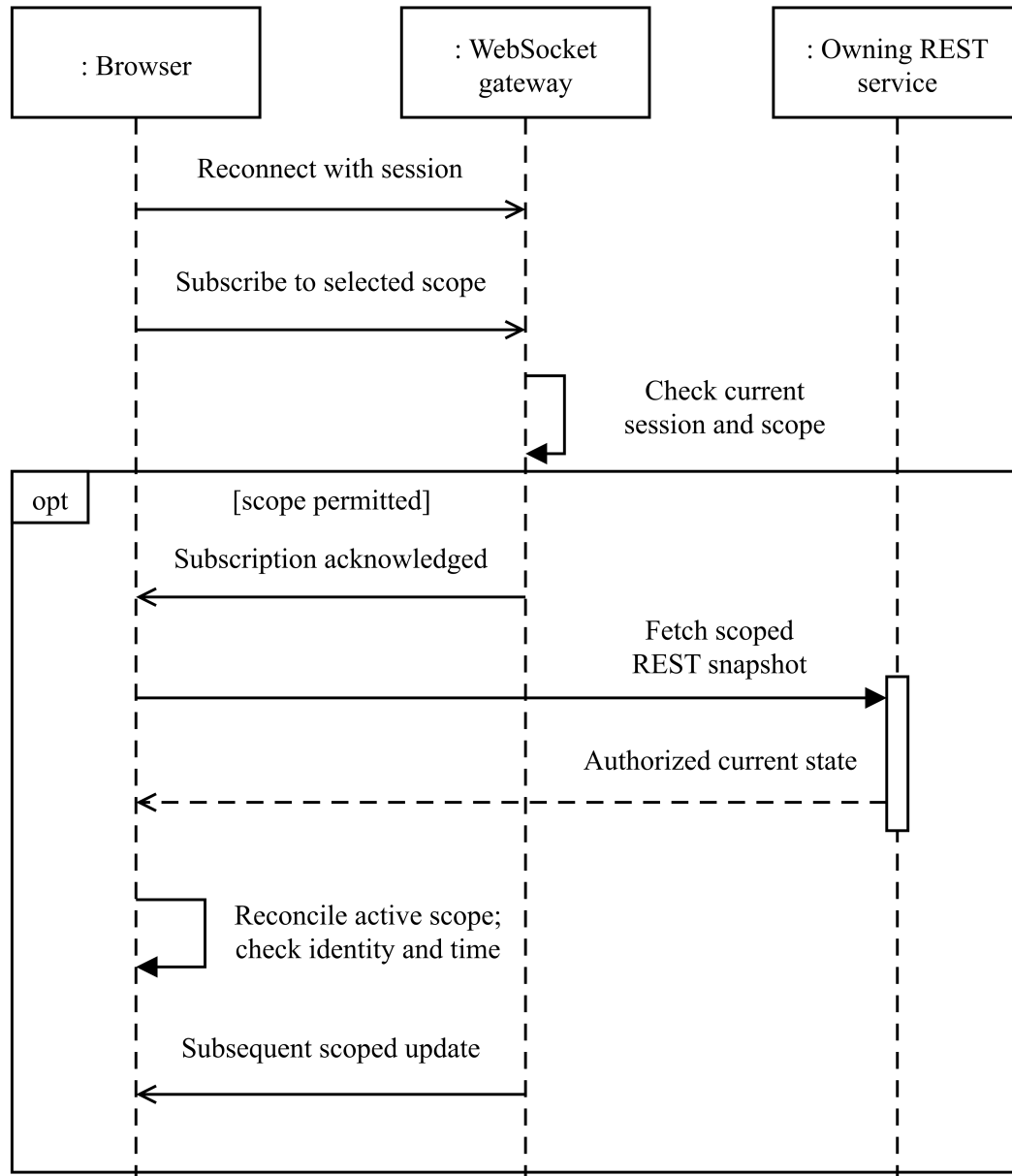
4.4.3 Interface and Event Design

The REST interface uses typed resource identifiers and the common JSON response envelope, with separate contracts for binary downloads and bodyless responses. Realtime messages provide updates within an authorized subscription, but the browser retains REST reconciliation as the means of reconstructing authoritative state after interruption. Client caches are partitioned by the relevant workspace and resource context, reducing the risk that a late response from an earlier selection is applied to the current view.

The domain-event envelope contains an event identifier, event type, timestamp, source, correlation identifier, version and payload. Event identity concerns logical delivery, while correlation connects related operations. The observation timestamp remains inside the telemetry payload because event publication time and measurement time are not interchangeable.

Delivery is designed around at-least-once processing. Consumers therefore require idempotent effects, and durable producers use an outbox to bridge a transaction and later publication. The presence of such an outbox in Intent, Simulation, Alert or Report does not imply that the telemetry source snapshot is also a durable observation queue. Each interface has its own failure and retention semantics, examined in Section 5.3.3.

Figure 4.9 illustrates the reconnect interaction required by IR-01–IR-02 and NFR-03. Dashed vertical lifelines distinguish the browser, WebSocket gateway and owning REST service. Solid open arrows show asynchronous socket messages, filled arrowheads show synchronous requests and dashed horizontal arrows show replies (Object Management Group, 2017).



If scope is denied: no ready state; show denial and retain no subscription.
Reconciliation applies only to the current workspace / network selection.

Figure 4.9. Authorized realtime reconnection and reconciliation

A subscription acknowledgement establishes readiness only for the selected authorized stream. The browser then requests current REST state and applies it within

the active workspace and resource context. Subsequent messages remain subject to identity and timestamp checks. Thus reconnection restores an authoritative view without treating socket delivery as a complete event history or allowing an earlier selection to overwrite the current one.

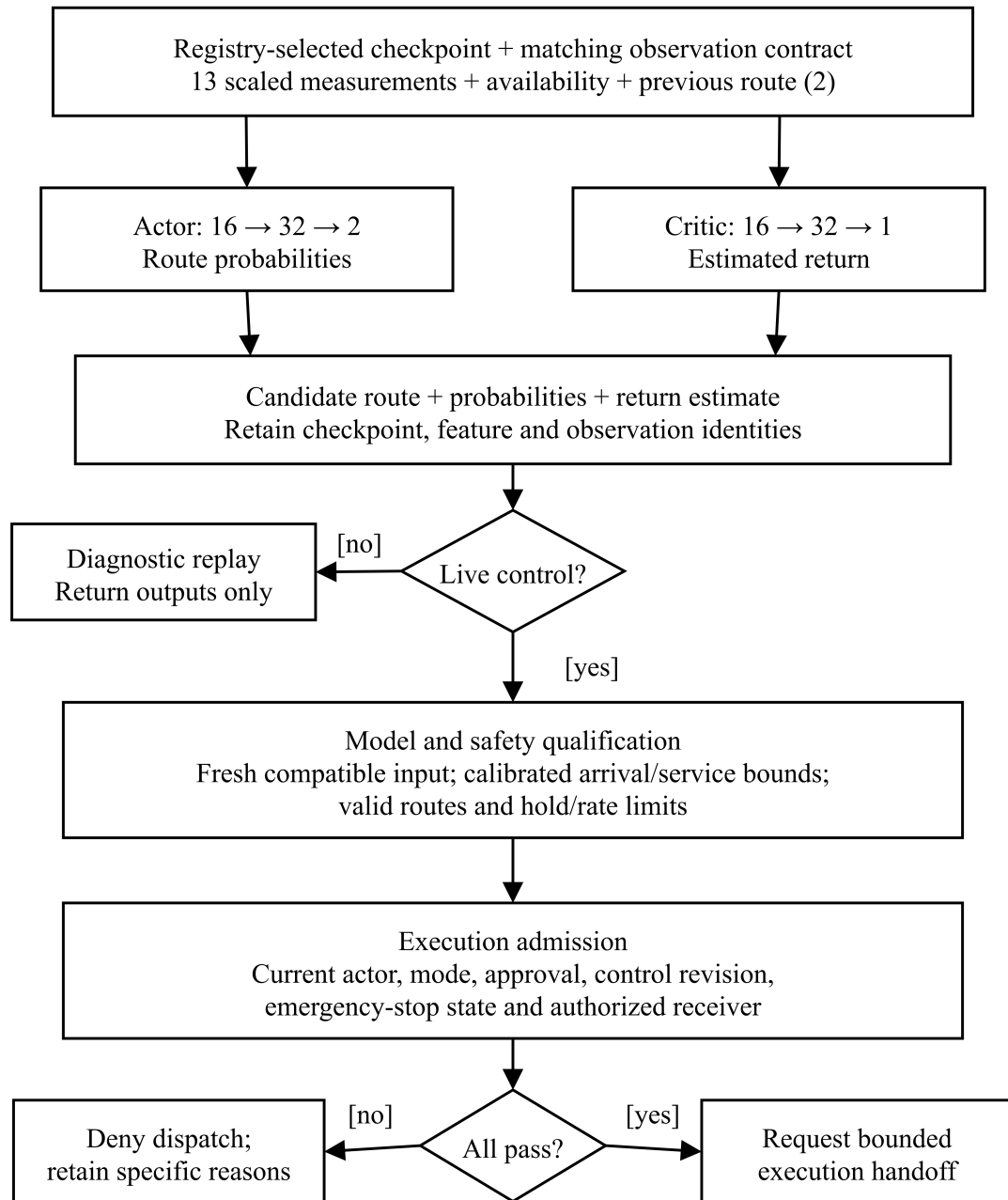
4.4.4 Learning Model and Safeguard Design

The learning component uses categorical PPO to choose between two candidate routes. Its sixteen-value observation combines thirteen scaled numerical measurements with a latency-availability indicator and a two-value representation of the previous action. Separate actor and critic networks each use a thirty-two-unit hidden layer. The actor returns route probabilities; the critic estimates return. The bounded action space makes the selected decision inspectable and keeps the initial experiment manageable.

The reward described in Chapter 3 favours useful delivery and penalizes delay, loss, utilization, queue pressure and route changes. The route-change term expresses a learning preference. It does not itself enforce a minimum path-holding interval or guarantee a queue bound. Those restrictions belong to the operational safeguard layer, which assesses valid routes, fresh observations, rate/hold limits and conditional arrival/service envelopes.

PPO's clipped training objective and operational safety should therefore remain separate concepts. The former constrains aspects of policy optimization; the latter depends on the physical meaning and validity of the supplied control evidence (Brunke et al., 2022; Schulman et al., 2017). RouteNet-Fermi likewise demonstrates why a useful network model still has prediction error that depends on evaluation conditions (Ferriol-Galmés et al., 2023). NANFO's design preserves these distinctions by retaining model provenance and denying dispatch when the required compatibility or calibration is absent.

Figure 4.10 separates the frozen PPO inference contract from the additional admission conditions of FR-12. Directed solid arrows show data or decision progression in this block-and-decision view. The diagram addresses FR-10, FR-12, NFR-04, NFR-07 and IR-04; it describes required control integration rather than a completed autonomous deployment.



Diagnostic inference and required live-control integration remain distinct.
Integrated autonomous acceptance is incomplete in the reported evaluation.

Figure 4.10. Frozen-model inference and autonomous admission design

The sixteen-value input contains thirteen scaled measurements, one latency-availability indicator and two previous-action values. The actor produces two route probabilities; the critic estimates return and does not select the route. A diagnostic replay ends with recorded outputs. Live autonomous dispatch additionally requires compatible fresh evidence, calibrated safeguards, current permission and a configured execution receiver. Missing prerequisites retain a denial reason, consistent with the incomplete integrated-autonomy acceptance reported in Chapter 5.

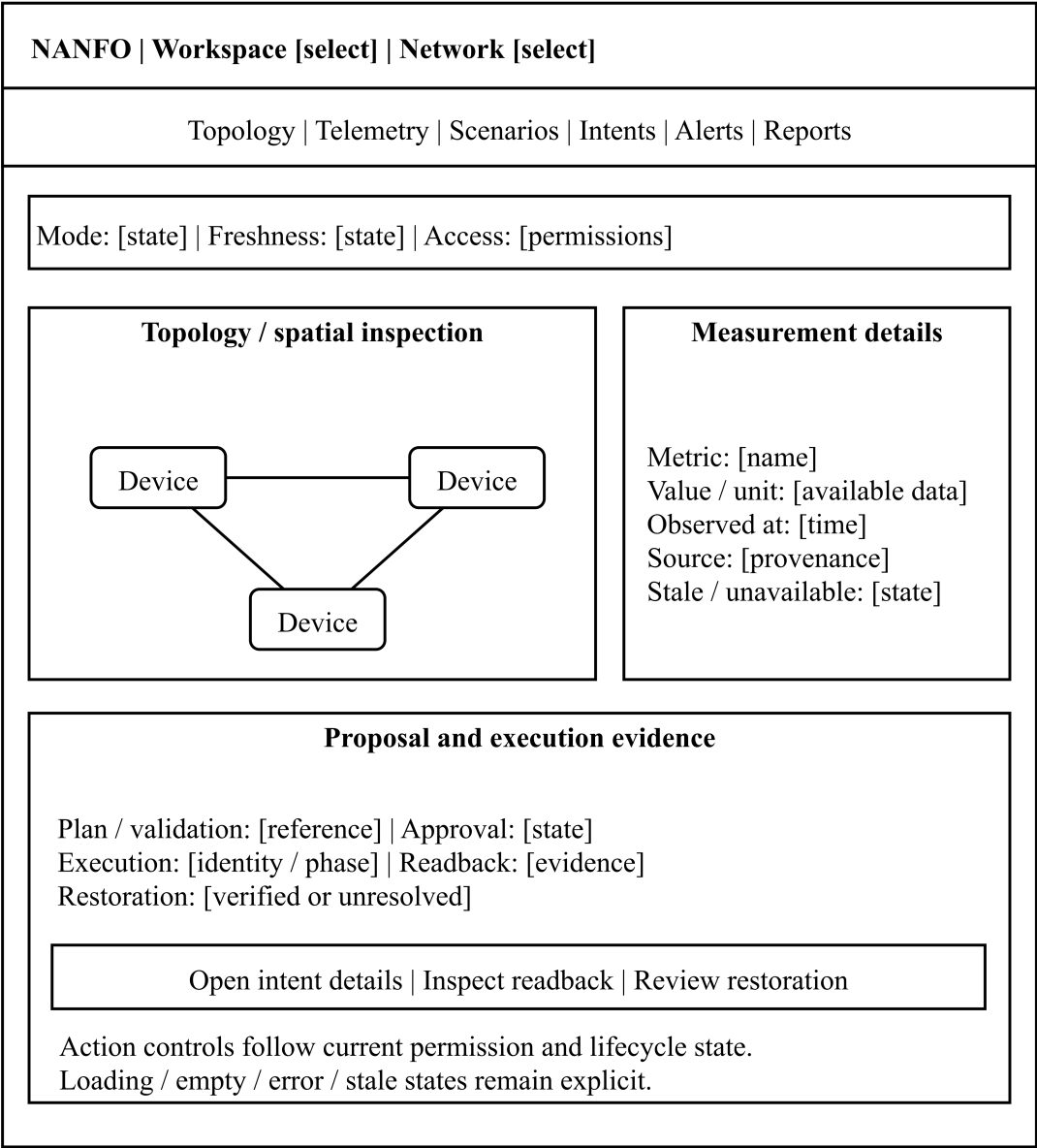
4.4.5 Operator Interaction Design

The interface follows the operator's task rather than the internal order of database tables. An operator selects a workspace and network, inspects topology and measurements, examines a scenario or proposed action, and then reviews the resulting execution evidence. Alerts and reports provide entry points for investigating a condition or presenting a retained result.

The digital-twin view is a spatial representation of the selected network context. It supports inspection, but visual proximity or a highlighted line is not proof that packets traversed a path. The interface consequently distinguishes approved configurations from selected-probe capture and separates modelled metrics from observed values. Timestamps, units, source information and stale-data states make those distinctions visible.

Forms and controls reflect current permissions, while the server remains authoritative. A countdown indicates the expected timing of an override, but restoration is complete only when the backend records the matching verification. Tabular alternatives, keyboard interaction and reduced-motion behaviour support inspection when a spatial view is unsuitable. Browser checks can establish that these controls function; user acceptance requires separate evidence from actual reviewers performing representative tasks.

Figure 4.11 is a low-fidelity wireframe of the operator information hierarchy described by NFR-06 and IR-01. It groups resource selection, topology inspection, measurement provenance and action evidence. The diagram is a design view with deliberately unpopulated values; it is not a screenshot or a record of a completed test.



Schematic navigation and information hierarchy; no sample result is asserted.

Figure 4.11. Operator evidence-view wireframe

Measurements are presented with units, source and observation time, while unavailable or stale data remains visibly distinct. The action area separates a proposed configuration from verified execution and recovery evidence. Navigation links lead to the relevant scenario, intent, alert and report workflows. The layout keeps permission

and execution state visible without allowing the presence of a button to imply server authorization.

4.4.6 Deployment and Recovery Design

The deployment package separates a frontend gateway, one API process, independent workers and private data stores. Persistent volumes hold relational data, graph data, Redis state, reports and laboratory handoff files. Trusted bindings and approved model artefacts are mounted separately so that the observation producer cannot modify its own authority or substitute a checkpoint.

The optional laboratory container is explicitly isolated from external network attachment. It contains the privileges required for network namespaces and Open vSwitch, while the API and workers do not receive a Docker socket or direct host-switch access. This is a trusted-code laboratory arrangement, not a guarantee that a privileged container can safely contain malicious code.

Recovery is designed as an ordered process: stop new admissions, establish the state of outstanding physical actions, quiesce writers, preserve persistent data and restore into the supported environment. Unresolved execution must not be forced into a successful state simply to complete a backup. The distinction between implemented recovery tooling and independently demonstrated full-stack restoration remains part of the evaluation in Chapter 5.

Figure 4.12 shows the deployment boundaries relevant to IR-03, IR-05 and NFR-11. In this deployment block view, rectangular containers identify host, process and storage boundaries; solid connectors represent communication paths or mounted file contracts. The optional privileged lab has no external network attachment and exchanges observations, commands and results through restricted shared volumes.

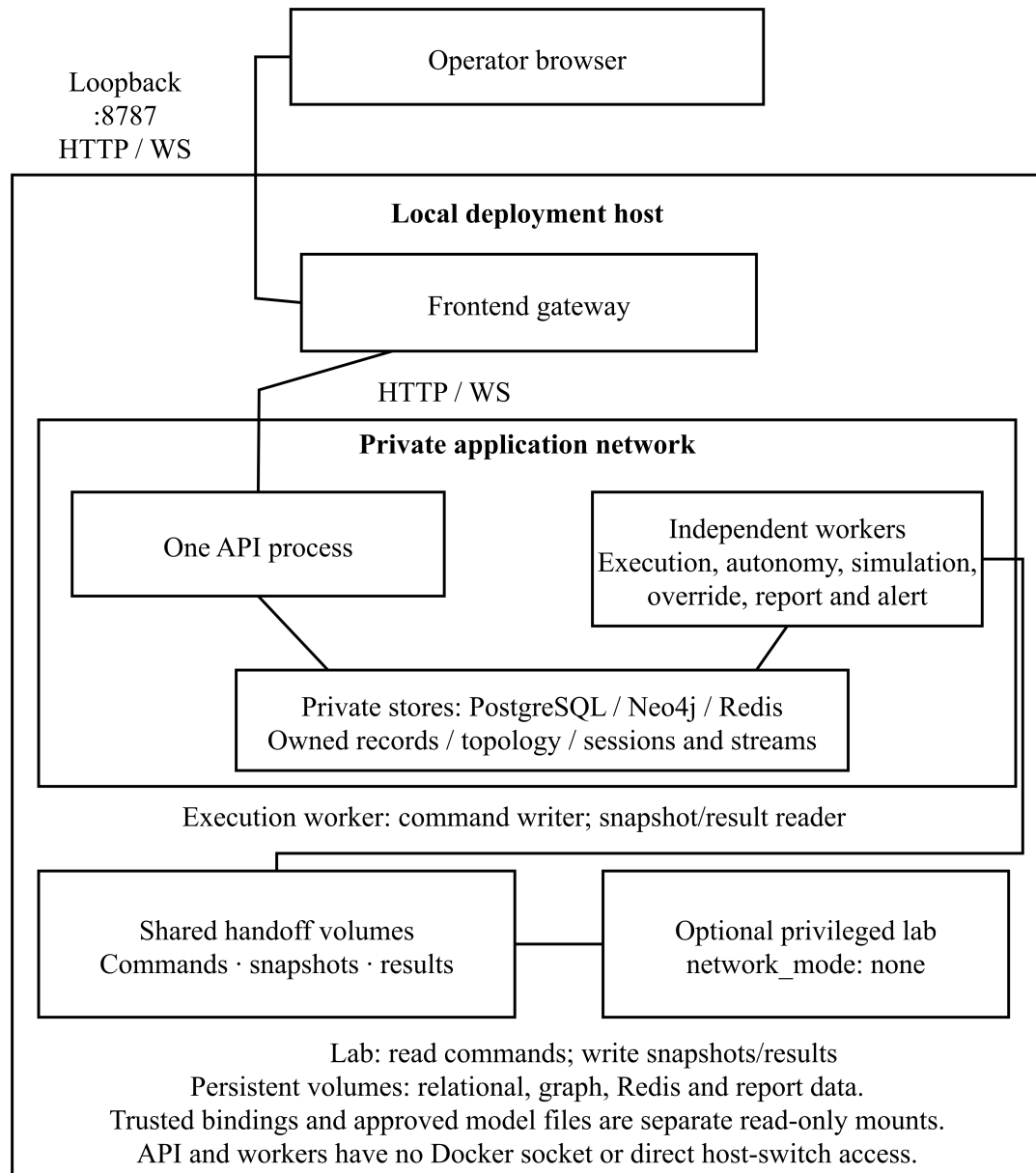


Figure 4.12. Deployment and laboratory file-handoff boundaries

The gateway exposes the local operator entry point, while the API, workers and data stores communicate on the private deployment network. The execution worker writes command files; the lab reads them and writes snapshots and results. Trusted bindings

are mounted separately from producer output. Recovery must preserve both persistent state and unresolved execution identity, and the diagram does not establish that a complete fresh-restore acceptance campaign has passed.

Chapter 5: System Implementation, Testing and Evaluation

5.1 Introduction

This chapter explains how NANFO was implemented and what the available evidence establishes about its operation. The discussion follows the system boundary and requirements defined in Chapter 4: authenticated access, measured network observations, model and simulation processing, controlled execution, recovery and reporting. It then evaluates the routing experiment and the application tests against their stated criteria.

NANFO is a hybrid software and machine-learning project whose observations come from an emulated network. The communication path therefore begins with switch counters, Linux queue measurements and endpoint probes rather than a physical sensor circuit. The implementation account follows each observation through the controller, application, storage and interface, and examines the consequences of interruption at those boundaries.

The evaluation draws on the engineering records and archived experiments identified in Section 5.5 (Bhudia, 2026). The evidence distinguishes network measurements, numerical-model outputs, controlled fixtures and software tests. The archived ADR-014 results comprise 240 decision windows across five routing methods, with separate model-selection and final-test records.

5.2 Implementation Environment

5.2.1 Hardware and Deployment Environment

Development and verification used a Linux host with Docker, network-namespace support and the kernel facilities required by Open vSwitch and traffic control. The bounded lab ran real virtual-switch and endpoint software inside a disposable container. This arrangement provided repeatable topology and traffic configuration while avoiding attachment to an operational campus network.

The resource limits in Table 5.1 describe the recorded laboratory configuration and the current deployment package. They are not all experimentally established minimum hardware requirements. In particular, the retained campaign summaries do not provide a complete physical-host processor and memory inventory. Replacing that

missing information with the present workstation's specification would not establish the equipment used in the historical experiment.

Table 5.1: Hardware, runtime and deployment conditions

Layer	Recorded configuration	Purpose and evidence boundary
Development/ verification host	Linux with Docker Engine; the initial live lab record identifies Docker 29.7.2.	Supported network namespaces, virtual Ethernet, kernel Open vSwitch and traffic control. Physical CPU model and total host RAM are not established by the retained campaign summaries.
Bounded network laboratory	Five forwarding nodes, four hosts, two candidate foreground paths; container limits of two CPUs, 768 MiB memory and 256 processes.	Limits taken from the lab configuration and recorded campaign scope, not a claim about minimum hardware for a campus deployment.
Application gateway	Loopback-bound HTTP entry point in the deployment package, forwarding through non-root nginx.	Provides local/private access; no public campus deployment is established.
Application API/workers	One supported realtime API owner and separate simulation, report, alert, execution and autonomy workers.	Separates request handling from long-running processing and recovery.
Persistent stores	PostgreSQL, Redis and Neo4j with private volumes; separate report and laboratory handoff storage.	Preserves transactional records, stream/session state, topology and controlled file exchange.

Layer	Recorded configuration	Purpose and evidence boundary
Deployment preflight	Approximately 6 GiB available host memory recommended by the package runbook; free-disk checks distinguish normal and optional-AI builds.	Operational installation guidance, not a measured minimum supporting a performance or availability guarantee.
Operator client	Browser workflows exercised through Chromium automation, including keyboard/mobile scenarios in the recorded frontend campaigns.	Browser automation establishes those tested workflows; minimum client hardware and field usability remain unmeasured.

The topology used nominal core/distribution capacities of 100 Mbps, a 50 Mbps distribution interconnection and 20 Mbps candidate access paths. Host-facing links were configured at 100 Mbps. Link delays ranged from one to five milliseconds per direction, and queues were configured with a 100-packet limit in the documented campus lab. The selected learning benchmark introduced a stationary 2/20 Mbps impairment pattern so that either candidate path could be the less suitable choice. These configured capacities were part of the experimental treatment, not measurements of a university's physical links.

The lab's isolation was an important condition of the implementation. Its container used no external Docker network, published port, host network namespace or physical network interface. Ryu listened on container loopback, and a local Unix socket supported operator-side coordination. The privileged lab contained trusted networking code; the application API and workers were not given equivalent host-control capabilities. Shared-host scheduling and kernel offload remained sources of measurement variation despite this isolation.

5.2.2 Software, Services and Configuration

Separate environments were retained for the controller, application backend and learning model. This was necessary because the controller depended on a legacy Python stack, while the backend and PyTorch runtime used different supported dependency sets. Installing every dependency into one environment would have obscured compatibility and made checkpoint reproduction more difficult.

Table 5.2 distinguishes versions recorded for the lab or verification environment from versions resolved by the current lockfiles and deployment build definitions. A lockfile identifies the intended dependency graph; it does not retroactively establish the runtime of every earlier test.

Table 5.2: Software versions and implementation roles

Component	Version or source-defined configuration	Role and evidence
Operator lab Python/Ryu	Python 3.9.23; Ryu 4.34; eventlet 0.30.2.	Versions recorded in the emulation runbook for the isolated controller environment.
Virtual networking	Mininet 2.3.0-1; Open vSwitch 2.15.0+ds1- 2+deb11u5.	Pinned laboratory packages; their legacy status limits production-security conclusions.
Backend Python	Python 3.14.7 recorded in report-worker verification; Python 3.12.14 pinned in the later deployment Dockerfile.	Distinct development/verification and packaged runtimes; the later image is not presented as the original campaign image.
Backend web/data libraries	FastAPI 0.115.14; Uvicorn 0.32.1; Pydantic 2.13.4; SQLAlchemy 2.0.52; asyncpg 0.30.0.	Current backend/poetry.lock; typed APIs, validation and asynchronous relational access.

Component	Version or source-defined configuration	Role and evidence
Authentication/ report libraries	bcrypt 4.3.0; ReportLab 4.4.10.	Current backend lockfile; password hashing and actual PDF generation. ReportLab 4.4.10 was also recorded in report verification.
Learning runtime	Python 3.12.14; CPU PyTorch 2.8.0; Gymnasium 1.2.0; NumPy 2.2.6; Pydantic 2.11.7.	The archived test plan records Python, PyTorch, Gymnasium and NumPy; the separate AI lockfile also resolves Pydantic.
Frontend	React 18.3.1; TypeScript 5.9.3; Vite 6.4.3; Three.js 0.170.0.	Current frontend/package-lock.json; operator interface, build and spatial rendering.
Client state	TanStack React Query 5.101.4; Zustand 5.0.15.	Current frontend lockfile; server-state queries and local interface state.
Test dependencies	pytest 8.4.2; Vitest 3.2.7; Playwright test package 1.62.1.	Current lockfile resolutions; historical result counts retain their campaign dates.
Packaged stores	PostgreSQL 17.6; Redis 7.4.5; Neo4j Community 5.26.12.	Digest-pinned deployment definitions; not a declaration that every earlier disposable test used these same images.
Build and gateway	Poetry 2.4.1; uv 0.8.22; Node.js 22.22.0; nginx 1.28.2.	Deployment Dockerfiles and build handoff record.

The backend image was built from an allowlisted source context, with a main-dependency installation and package-consistency check. The frontend build used the lockfile-driven installation, linting, type checking and a production build. The

optional AI image installed its own environment without embedding a model checkpoint or granting execution authority. The deployment record reports successful source builds and import checks, while explicitly leaving complete model inference and full-stack recovery acceptance separate.

Configuration also determined what the application was capable of doing. Selecting an execution mode did not install a controller, evaluator or qualified model. The packaged defaults left laboratory control and diagnostics unavailable until their trusted bindings, input artefacts and relevant settings were supplied. This avoided converting an environment label into a false claim of operational readiness.

5.2.3 Deployment and Reproducibility

The deployment procedure created a new named instance and private state directory, generated local secrets, built the required images and initialized fresh storage. The initializer separated database ownership/migration privileges from the runtime account. It refused an existing populated schema rather than silently reinitializing a working installation. The intended sequence was initialization, configuration inspection, image build, application start and status verification.

The runbook's fresh-instance example begins with `python3 deploy/manage.py --state /srv/nanfo/instance1 init --project nanfo-deploy-instance1 --email operator@example.com --port 8787`. The operator then invokes `config`, `build`, `start` and `status` through the same script and state-directory argument. These example names identify a new instance rather than an existing verification deployment. The bootstrap password is read locally from its protected secret file, and the initial application is reached at the configured loopback gateway. The laboratory and frozen-model overlays are enabled separately after their bindings and artefacts have been provisioned.

Only the gateway published a loopback host port. The API and workers used private datastore connections, non-root execution, read-only roots and constrained writable mounts. The deployment record identified separate volumes for PostgreSQL, Redis, Neo4j, reports and the lab's output, commands and results. The execution worker could write commands, while the API's access to those control files was read-only. The producer did not receive the trusted binding or model registry as writable data.

Readiness and liveness were checked separately. The liveness endpoint indicated that a process responded; readiness examined the expected schema, datastore connectivity, API lease and event-consumer state. Worker health reflected completed loop progress rather than process existence alone. These distinctions matter when an application is running but cannot safely accept a new execution or produce a report.

Reproducing the archived routing result requires more than rebuilding the latest images. The selected checkpoint must be paired with its frozen source, runtime, feature contract, workload plan, raw response traces and selection record. The checkpoint digest matched the retained archive during the documentation check. Preserving those files remains necessary because the current deployment image is not a substitute for the original frozen experiment image.

5.3 System Implementation

5.3.1 Identity, Resource Scope and Application Data

The Identity module implemented authentication with bcrypt password hashes and signed access/refresh tokens. The configured bcrypt work factor was twelve. Tokens identified a server-managed session family, and the session repository supported atomic refresh rotation. Replay of a rotated refresh token revoked the family, while logout invalidated the session. Authentication rechecked active user status and current permissions, rather than relying only on the role list embedded when the token was issued.

Organization membership supplied another authorization boundary. A valid global capability did not automatically grant access to every workspace. Protected services resolved the current resource scope and membership before reading or mutating data. The frontend reflected permitted controls, but it could not grant itself authority by displaying a button or modifying a request. These mechanisms implemented FR-01, FR-02 and NFR-01.

Inventory operations returned the network and device identifiers later used by the telemetry binder. Organization, Network and Telemetry retained their own repository responsibilities. Within-module relationships, such as device-to-network ownership, used relational integrity constraints. Cross-module references were checked through owning services. This structure connected the logical design in Section 4.4.2 to the

implementation without assuming that every relationship was a physical SQL foreign key.

5.3.2 Emulated Devices and Measurement Acquisition

The observation source combined OpenFlow statistics, Linux traffic-control information and endpoint probes. Ryu polled switches approximately once per second. Port and flow replies were correlated by switch, transaction identifier and reply type, and complete multipart replies were required before a sample was accepted. Incomplete reply rounds expired rather than being converted into zero-valued measurements.

Queue measurements came from the leaf netem queue discipline. The parser retained backlog bytes and packet count without adding the parent HTB backlog, which represents the same queued traffic. This avoided double counting. Two-packet ICMP probes were normally issued every five seconds, retaining transmitted/received counts, mean round-trip time and the actual measurement interval. An unanswered probe had no measured RTT.

Topology discovery used observed LLDP relationships, while host attachments were recognized only after matching traffic arrived at the expected host-facing ingress. This prevented a transit packet from being interpreted as a new endpoint attachment. Unknown forwarding cases did not receive an unrestricted flood or normal-switching fallback. The resulting topology was suitable for the bounded experiment, although discovery and connectivity did not establish the fidelity of a physical campus model.

5.3.3 Communication, Storage and Display

5.3.3.1 Snapshot and Message Format

The producer wrote a versioned snapshot containing version, topology_id, run_id, sequence, observed_at, and arrays of switches, links, hosts, queues and probes. Switch records contained port and flow statistics; queue records identified the switch, port, observation time and backlog; probe records identified the endpoint pair, transmitted/received counts, average RTT or null, and interval. These were schema fields from the implemented contract, not values fabricated for an example reading.

Snapshot publication used a same-directory temporary file, file synchronization, atomic replacement and directory synchronization. The file was bounded to 1 MiB. Atomic replacement protected readers from a partially written snapshot, but the producer replaced one latest-state file rather than appending a durable observation history. A new run identifier distinguished a restarted laboratory from the preceding run.

After validation and resource binding, the backend produced normalized metric samples. Each sample retained the canonical device, network and workspace, its source observation timestamp, metric name, numerical value, unit and provenance tags. The domain-event envelope added event_id, event_type, publication timestamp, source, correlation_id, version and the serialized payload. Publication time and measurement time remained distinct.

5.3.3.2 Identity and Trust at the Device Boundary

The lab did not implement certificate-based authentication for every virtual switch. A datapath identifier, MAC address or LLDP observation identified a resource but did not by itself establish cryptographic device identity. Trust instead depended on the isolated, operator-controlled lab and a binding created through authenticated application inventory operations.

The binding mapped expected emulator identities to actual API-returned resource identifiers. It was stored outside the producer's writable output directory. Before accepting a fresh poll or retry, the backend checked the binding's actor, current membership and permitted resources. The producer could therefore report measurements but could not legitimately move them into an arbitrary tenant by changing its own snapshot. This is an application/laboratory trust arrangement whose limits must be reconsidered before connection to untrusted physical devices.

5.3.3.3 Rate Derivation and Persistence

Port rates were derived from consecutive byte-counter values and actual OpenFlow port-duration differences. The first valid counter sample established a baseline; it did not produce a fabricated throughput or utilization value. A counter reset or invalid interval invalidated the corresponding rate calculation. Trusted configured capacity supplied the denominator for utilization.

Stable observation identities incorporated the run, bound device, metric dimensions and observation time. Rate identities also retained their interval boundaries. This corrected an earlier approach in which snapshot sequence changes could give repeated cached observations different identities. PostgreSQL uniqueness then prevented a retried event from becoming a second logical telemetry record.

The alert integration operated after Telemetry persistence succeeded. It treated the incoming event as a locator and reread the accepted observation through the Telemetry service, checking the matching event identity and source. This prevented altered replay payloads from becoming new evidence. Alert state, history and its lifecycle outbox committed together under the module's own transaction.

5.3.3.4 Buffering, Retry and Reconnection

The telemetry adapter retained a pending batch until publication of the complete batch was acknowledged. It did not advance to a newer snapshot while part of the current batch remained unpublished. Replaying the full batch was safe for observations already persisted because their identities remained stable. The documented live test exercised a published prefix of 102 samples followed by replay of the complete 204-sample batch after the producer had advanced.

This pending batch was held in memory. A collector-process restart could therefore discard it, and an expired retained batch required explicit recovery instead of being silently replaced by a newer snapshot. Atomic snapshot publication and downstream deduplication did not provide a loss-free durable queue between every source measurement and the database. This limitation is central to DR-06 and NFR-11.

For events already in Redis Streams, consumer groups supported pending-entry handling, success markers and reclaim. Poison-message handling recorded a dead-letter outcome before acknowledgement. Durable outboxes in the relevant workflow modules retained event identity when publication was retried. These mechanisms reduced duplicate effects, but the overall transport remained at-least-once; there was no system-wide exactly-once guarantee.

The browser subscribed within its authenticated resource context. Subscription acknowledgement established readiness, and reconnect/backpressure handling initiated bounded REST reconciliation. Source timestamps and revision/epoch checks

prevented older responses from replacing newer context. The digital twin marked stale observations and expired its congestion indication after sixty seconds rather than continuously treating a disconnected source's last value as current.

5.3.3.5 Disconnection and Power-Recovery Boundaries

Controller or link interruption could invalidate a preparation check, interrupt measurement or require restoration. Negative laboratory cases exercised link-down, stale-controller and controller-disconnect conditions, followed by independently observed probe recovery. Worker and lab restart tests examined command identity, cancellation and restoration. These are specific fault cases, not proof that every interruption is lossless.

A full host power loss affects a broader chain: the producer's latest snapshot, memory-only pending batches, Redis persistence, relational commits, command files and the lab journal. The deployment package enabled persistent store volumes and explicit backup/restore operations, but the retained record does not certify a complete power-loss recovery test across all components. Consequently, the system cannot claim that every reading collected before an outage reappears exactly once after power restoration.

5.3.4 Numerical Simulation and Controlled Network Actions

The simulation module implemented a deterministic finite-buffer fluid model. Scenarios specified directed links, capacities, buffers, demand schedules, routes, timestep and horizon. Each checkpoint accounted for offered traffic as delivered, dropped, queued or in flight. A completed calculation could still fail its objective limits, so computational completion and scenario acceptance were separate outcomes.

The reported analytic example offered 2 Mbps to a 1 Mbps link for one modelled second. It delivered 125,000 bytes, dropped 112,500 bytes and retained 12,500 bytes in the queue. The components sum to 250,000 offered bytes. The reported model latency of 180.0195 ms belonged to this numerical scenario; it was not an observed ICMP RTT. This example verified accounting under declared model assumptions, supporting FR-06 and NFR-08 without claiming packet-level fidelity.

Manual execution implemented rerouting, supported SELECT multipath, DSCP-aware shaping and policing, and restoration within the bounded lab. The worker used

the protected mailbox rather than calling Docker or a host-switch CLI. The lab owned OpenFlow and traffic-control operations, its journal and the matching result. Current actor, binding, plan and timing checks preceded dispatch.

Recorded workload tests measured approximately 18.8 Mbps baseline TCP delivery and 37.8 Mbps with the supported multipath configuration. A 5 Mbps shaping case delivered approximately 4.76 Mbps for classified traffic, while the corresponding policing case delivered approximately 4.87 Mbps. Unclassified and restored traffic remained near 17.71 Mbps in the recorded UDP tests. These measurements establish the behaviour of the tested manual controls; they do not mean the PPO action space included shaping, policing or continuous flow-split optimization.

5.3.5 Model Training, Selection and Diagnostic Inference

5.3.5.1 Observation and Policy Implementation

The implemented model used thirteen numerical features, a latency-availability indicator and two previous-action indicators. Numerical features were transformed by fixed logarithmic scaling. The model contained separate thirty-two-unit actor and critic hidden layers; the actor selected one of two paths, and the critic estimated expected return. The categorical decision rule matched the bounded routing problem rather than a continuous link-weight vector.

The reward combined useful delivery with penalties for RTT, loss, utilization, queue occupancy and route changes, as specified in Chapter 3. Actual offered foreground rate was used in the delivery ratio. A verified zero-reply outage retained null observed latency, with a separately labelled censored-delay reward penalty. Incomplete or inconsistent windows were excluded from optimization. This preserved the distinction between a learning convention and an instrument reading.

One archived PPO decision window illustrates the measurement-to-model boundary. The foreground sender recorded 2,052 packets, and the receiver recorded 1,843 packets containing 2,211,600 payload bytes. Dividing those received bytes by the sender's 3.38195343-second duration gave 5.231532712 Mbps goodput; the corresponding delivery loss was 10.185185%. The independent ICMP observation reported 58.629 ms mean RTT. These fields entered the observation record under their distinct meanings. They describe one transition window, not the campaign

average, and show why sender duration, receiver bytes and probe results must remain separately identifiable.

PPO supplied the clipped policy objective, with advantage estimation, value loss, entropy regularization and gradient clipping (Schulman et al., 2017). These optimizer controls supported training but did not qualify a live route as safe. The operational assessment required its own compatible observation, route and calibrated arrival/service bounds.

5.3.5.2 Training and Selection Chronology

The ADR-014 training campaign used model seed 44, a learning rate of 0.003, discount factor 0.9, generalized-advantage parameter 0.95, sixteen-transition rollouts and four optimization epochs. These settings were fixed before collection. The rationale was to obtain measurable policy movement within the bounded on-policy budget while discounting distant noisy returns in four-decision stationary episodes. No post-test hyperparameter search was reported.

The campaign collected 384 training transitions from seeds 1600–1695, producing 24 updates over six continuation blocks. Validation used eight separate workload seeds, 2700–2707, at 128, 256 and 384 transitions. The twelve reserved test seeds, 3900–3911, were not used to update weights. The total training-stage collection included calibration and validation sessions as well as training, so its 624 measured decisions must not be presented as 624 training transitions.

The original campaign stopped before its planned minimum of 512 transitions because the remaining-budget admission rule reserved time for further work and cleanup. The 384-transition policy demonstrated useful directionality but was not eligible under that original selection rule. A separately authorized test-only continuation later selected the best existing validation-qualified checkpoint before opening the reserved test. The selection exception was recorded; the original plan and early-stop outcome were not rewritten.

This chronology matters because model quality, selection eligibility and final evaluation are different decisions. The test result may be reported with its declared exception, but it cannot be described as an uninterrupted campaign that completed the

original 512-transition plan. It also cannot support an assertion that a model was chosen after examining its final-test performance.

5.3.5.3 Frozen Inference and Operational Integration

The selected checkpoint was retained with its model and source identities. Recorded fresh-process checks reproduced the same action, probabilities and value under matching inputs. The application later exposed historical diagnostic inference through a constrained runtime, with Landlock and seccomp limiting access to the allowed model files and private scratch space. Tests of same-user control-file access and filesystem escape were denied in the recorded environment.

Diagnostics were deliberately different from live autonomous control. A caller selected an allowed checkpoint and historical observation; it did not supply an arbitrary executable path or gain authority to act on the network. Requested reward-setting changes were retained as requests requiring retraining, while the effective frozen checkpoint remained unchanged. FR-10 therefore had evidence independent of the still-open FR-12 integrated-autonomy requirement.

5.3.6 Alerts, Reports and Operator Evidence

The measured-alert detector used four metric families: utilization, ICMP RTT, probe loss and queue packet backlog. The default breach/recovery thresholds were 85/70 percent utilization, 100/70 ms RTT, 2/1 percent probe loss and 80/40 queued packets. In the documented 100-packet queue configuration, the queue threshold also corresponds to 80/40 percent of capacity; the implementation itself stores the packet thresholds.

Both breach and recovery required at least three distinct ordered observations spanning ten seconds. A gap greater than ten seconds reset the window, and observations older than thirty seconds were ineligible. An incident could not be resolved merely because no further readings arrived. Detector identity included resource scope, run, metric, source and rule version, preventing one run's recovery from silently resolving another run's incident.

The Report module generated real CSV and PDF bytes from authorized, bounded source snapshots. A worker owned rendering and artifact storage; the API served authorized downloads. Recorded byte counts and SHA-256 digests supported integrity

checks, and the interface verified downloaded content by default. A snapshot's known omissions remained visible instead of being disguised as a complete account of every network event.

The operator interface brought together topology, telemetry, simulations, intents, diagnostics, alerts and reports. It displayed the difference between requested and effective configuration and between approved paths and selected-probe capture. The plugin registry managed metadata rather than executable extensions. The energy function estimated consumption under declared assumptions and did not measure physical savings or shut down ports. These scope boundaries prevented supporting interface modules from becoming unsupported claims about additional research achievements.

5.4 Testing and Evaluation

5.4.1 Test Strategy and Acceptance Criteria

Testing combined component checks, integration tests, complete application workflows and measured network experiments. Unit tests examined calculations, validation and state transitions. Integration tests examined module contracts, transactions and service boundaries. System tests followed authenticated workflows through independent workers and the actual lab where applicable. Browser automation examined interface behaviour, while model evaluation used separate workload partitions and comparison rules.

These test levels answer different questions. Black-box, white-box and grey-box techniques describe the knowledge available to a test, rather than replacing the levels themselves. ISO/IEC/IEEE 29119-1:2022 provides general testing concepts supporting this distinction (International Organization for Standardization et al., 2022). The documentation uses those concepts to distinguish test design, execution, incident handling and reported outcomes, without claiming certification against the complete standard.

The acceptance evidence is grouped into measured, modelled and fixture-based results. A measured routing test can establish behaviour in the emulator. A numerical accounting test can establish a property of the simulator. A browser test backed by

mocked APIs can establish interface behaviour under that contract. None should be substituted for another simply because all return a passing status.

For the routing benchmark, the predeclared primary criterion required a positive lower 95% confidence bound for PPO-minus-OSPF goodput and a negative upper bound for the corresponding RTT difference. The pairing unit was the workload seed, with four decisions averaged within each seed. The twelve seed means were not replaced by packet counts or 48 dependent windows to create an artificially large sample. This treatment reflects the uncertainty concerns raised by Agarwal et al. (2021).

For application testing, acceptance required the expected authorization, identity, lifecycle and evidence outcome. Examples included one durable execution under concurrent retries, no duplicate telemetry records after replay, no false completion after interrupted capture, and matching artifact hashes after report download. Relevant negative cases covered stale data, invalid identity, missing authority, incompatible evidence and uncertain compensation. OWASP's Web Security Testing Guide informed the separation of authentication, authorization, session-management, input-validation and business-logic concerns (OWASP Foundation, 2020). The retained tests do not constitute a comprehensive external penetration test.

5.4.2 Test Cases, Defects and Retesting

Table 5.3 presents the principal cases and the available outcomes. “Recorded pass” means the retained project record reports that the case passed; it does not claim that the case was rerun while preparing this chapter. The ADR-014 delivery statistics additionally passed the read-only archive reconstruction described in Section 5.1. Where a record covers fixtures or a model, that scope is stated. The evidence identifiers resolve to exact project records in Table 5.9.

Table 5.3: Test cases and recorded results

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-01 — FR-01, FR-02, NFR-01	Session rotation, replay, logout, read-only and cross-tenant cases in the foundation campaign.	Invalid/revoked sessions and unauthorized scopes are denied; permitted sessions retain only current access.	Foundation regression reported 1,112 backend tests passing; session atomicity used fakeredis and repository/transport fixtures. E01.	Recorded pass within fixture scope.
TC-02 — FR-01–FR-03, IR-03	Real authenticated inventory creation, binding and later actor disable in the observation verifier.	API-returned identities map to observed resources; disabled actor cannot continue collection.	Nine bound devices were observed, and subsequent collection was denied after actor disable. E02.	Recorded live pass.
TC-03 — FR-04, DR-01–DR-03	Final integrated telemetry retest with real API, stores and WebSocket.	Accepted source observations match stored records and delivered frames after the bounded drain.	2,178 unique observations/rows/frames; zero duplicates and zero final backlog/pending entries; nine nodes and eleven edges. E06.	Recorded live pass.

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-04 — FR-04, NFR-02, IR-03	Partial publication of 102 of 204 samples, producer advancement, then complete replay.	Previously stored prefix is not duplicated and missing suffix is accepted.	Corrected live verifier reported successful replay without duplicate logical rows. Pending batch remained memory-only. E02.	Recorded live pass for replay; no durable-source guarantee.
TC-05 — FR-04, DR-02	Independent controller/OVS counter and queue checks.	Port-rate derivation matches counter-duration evidence; leaf backlog is not counted twice.	Corrected records matched independent counter calculations. Final Step14 peaks included 19.943285 Mbps port throughput and 100 queued packets. E02, E06.	Recorded measured pass.
TC-06 — FR-05, NFR-03	Sustained measured utilization breach and recovery under actual generated traffic.	Incident appears only after the configured sustained breach and resolves after qualified recovery; replay preserves identity.	Twenty-six observations; breach span 11.024056 s and recovery span 12.031185 s; duplicate replay left incident/history identities unchanged. E06.	Recorded live pass.
TC-07 — FR-06, NFR-08	Configured analytic bottleneck and deterministic lifecycle tests.	Traffic accounting balances; replay/pause/continuation preserves the declared model result.	250,000 offered bytes accounted for as 125,000 delivered, 112,500 dropped and 12,500 queued in the reported example. Nineteen simulation/dispatch and four intent database tests passed. E04.	Recorded model/integration pass.

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-08 — FR-07, FR-08	Authenticated manual reroute, multipath and bounded QoS workloads.	Approved configuration is applied and read back; workload effects correspond to the selected operation.	Approximately 18.8 Mbps baseline, 37.8 Mbps multipath, 4.76 Mbps shaped and 4.87 Mbps policed traffic; independent restore workloads recorded. E03, E06.	Recorded live pass.
TC-09 — FR-08, FR-09, NFR-02	Four concurrent execution submissions and worker death/lost-result recovery.	One durable execution; recovery retains the original command rather than repeating a new action.	One execution for four submissions; command identity and journal action count preserved. E03.	Recorded live pass.
TC-10 — FR-09, NFR-05	Six override cases involving expiry, restart, revocation and STOP/capture overlap.	Owened flows are restored and exact cancellation identity is retained; interrupted capture is not labelled complete.	All six retest cases restored owned flows to zero; verification intervals ranged from 6.077202 to 9.171165 s. E06.	Recorded live pass in six bounded cases.

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-11 — FR-09, NFR-07	Link-down, stale-controller and controller-disconnect fault cases with deliberate eight-second dwell.	Unsafe preparation is rejected and recovered probes demonstrate restored reachability.	Eight of eight recovered probes per case; fault-to-recovered-probe intervals 8.8623, 9.7438 and 11.8544 s, including dwell. E06.	Recorded live pass; not a production recovery-time guarantee.
TC-12 — FR-10, NFR-04, IR-04	Frozen historical inference in independent confined processes.	Compatible input reproduces recorded outputs; prohibited control-file and filesystem access is denied.	Action 0, probabilities approximately [0.9718289375, 0.0281710345] and value 3.2172513 reproduced in the recorded test. E05, E08.	Recorded inference/confine ment pass.
TC-13 — FR-11, NFR-01, DR-05	Report rendering, parsing, owner download, tamper and tenant-denial cases.	Actual artifact bytes match metadata; unauthorized or altered artifacts are rejected.	PDF/CSV parsing, size/hash and access checks passed; report source records were controlled fixtures. E06.	Recorded pipeline pass with fixture data.

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-14 — NFR-09, DR-04	Five-method ADR-014 held-out comparison; twelve seeds, four decisions per seed and frozen policy.	PPO meets the predeclared goodput/RTT criterion against nominal-cost OSPF.	Paired goodput interval [0.663966, 3.287611] Mbps; RTT interval [−139.991071, −17.880054] ms. Read-only reconstruction matched the archive. E08.	Recorded benchmark pass; statistics cross-checked within the impairment scope.
TC-15 — FR-12, NFR-07	Attempted autonomous dispatch without the required compatible model/calibration/authority.	Dispatch is denied while missing prerequisites remain visible.	Governed-autonomy records report fail-closed rejection and open activation gates. E10.	Recorded rejection pass; autonomous efficacy not established.
TC-16 — FR-12, NFR-09	Integrated PPO plus safeguards across the planned normal/burst/sustained workload extension.	Matched live evidence establishes delivery, route stability and early-intervention behaviour.	DRL-plus-safety and repeated held-out acceptance remained blocked in the consolidated campaign. E06.	Blocked; no result claimed.

Test ID and traceability	Scenario, data and conditions	Expected result	Actual result and evidence	Verdict
TC-17 — NFR-06	Recorded keyboard/mobile/reconnect and frontend contract workflows.	Permitted controls work and stale/modelled/observed states remain distinguishable.	Consolidated frontend record: 487 unit/component tests and 41 no-retry browser tests passed; not every browser case used a live backend. E06, E11.	Recorded interface pass within stated test conditions.
TC-18 — NFR-11, DR-06	Complete cold backup, fresh restore and host-power-loss recovery.	Persistent evidence and workflow state recover under the declared procedure.	Forty-eight isolated operations tests were recorded, but complete live recovery acceptance was not established. E12.	Full-stack acceptance unverified.
TC-19 — NFR-06	Representative administrators perform defined tasks and evaluate usability/fitness.	Participant tasks, completion, errors and feedback support an acceptance conclusion.	No participant-based UAT dataset was supplied in the reviewed evidence.	Not performed in the available record.

The consolidated Step14 campaign initially recorded 18 passing cases, four failing cases and nine blocked cases. After diagnosis and focused retesting, it recorded 22 passes, zero remaining failures among those executed cases, and nine blocked cases. The final “partial” status is therefore appropriate. Dividing 22 passes by the 22 executed cases would conceal the blocked acceptance work and should not be reported as complete system acceptance.

Table 5.4 records representative defects and methodological corrections. These cases show why test incidents were retained rather than removed after a later successful run.

Table 5.4: Defects, corrections and retest evidence

Incident	Original effect	Correction	Retest and interpretation
Observation identity/rate derivation	Cached observations could acquire new identities; rate timing used reply completion rather than port duration.	Stable observation/interval identity and actual port-duration differences.	Corrected source/storage/replay checks passed. Earlier measurement claims were superseded, not treated as equivalent evidence. E02.
INET event serialization	A created host could fail to reach the topology event path because the returned database address object was not JSON serializable.	Serialize the address as a string while preserving null.	Live host creation projected all four hosts through the event consumer. E02.
Cancellation reconciliation	A failed receipt with verified rollback was rejected while cancellation was pending, leaving execution indefinitely uncertain.	Accept matching, time-valid failed or cancelled terminal evidence when compensation/no-mutation is verified.	The new positive regression failed before the correction; live deadline compensation subsequently passed. E06.
STOP/capture verifier race	Capture could finish before the intended STOP/recovery ordering was exercised.	Control only the verifier's own worker scheduling and observe actual capture children before committing STOP.	All six override cases passed without altering database time or relaxing assertions. E06.
Telemetry verifier dispatch/backlog	Legacy handler invocation and serial verification produced an incomplete drain at the equality assertion.	Use the actual event-processing contract with bounded independent-series concurrency and preserved FIFO ordering.	Final 2,178 observations matched with zero lag/pending entries under the unchanged forty-second drain bound. E06.

Incident	Original effect	Correction	Retest and interpretation
Training-budget/ selection shortfall	Training stopped at 384 transitions before the original 512-transition selection minimum.	Preserve the original stop and record a separately authorized test-only selection exception.	Reserved tests were opened only after the later checkpoint selection; the original campaign was not relabelled as completed to plan. E07, E08.

5.4.3 Project-Specific Evaluation

5.4.3.1 Held-Out Routing Performance

The five routing methods were evaluated once on the same twelve reserved workload seeds, with six seeds for each stationary impairment direction. Each episode contributed four measured decisions, giving 48 decision windows per method and 240 across all methods. Sixty reset windows were excluded from the reported performance metrics. The recorded run contained no invalid windows or retries and reported no service outage for the evaluated methods.

Table 5.5 presents the retained mean outcomes. Loss is the mean per-decision-window verified-drain loss, rather than a ratio obtained by pooling all packets. Route changes are counts within the reported evaluation and should not be interpreted as a universal switching rate.

Table 5.5: Recorded ADR-014 held-out routing results

Method	Goodput (Mbps)	ICMP RTT (ms)	Mean window loss (%)	Route changes
PPO	5.921743	25.154896	1.353850	6
Constant route 0	3.933795	158.371667	34.403610	0
Constant route 1	4.163708	168.454771	30.682101	12
Load-aware heuristic	5.152365	77.230354	14.185030	29
Nominal-cost OSPF	3.945954	104.090458	34.202956	0

Source: retained ADR-014 holdout record, E08 (Bhudia, 2026).

PPO selected the less impaired path in both directional conditions. It made 24 decisions on each route. Its six changes occurred on the first decision of the six path-0-impaired episodes, moving from the reset route to the alternative. Constant route 1 changed from the route-0 reset once in each of twelve episodes, explaining its twelve counted changes despite being constant within an episode. This definition is necessary to interpret the stability column correctly.

Table 5.6 gives paired differences using the twelve seed means. The predeclared OSPF criterion was satisfied because the entire goodput interval was positive and the entire RTT interval was negative. The other comparisons provide additional evidence but do not replace the originally declared primary comparison.

Table 5.6: PPO-minus-baseline paired differences and 95% confidence intervals

Comparator	Goodput difference, Mbps [95% CI]	RTT difference, ms [95% CI]
Nominal-cost OSPF	1.975789 [0.663966, 3.287611]	−78.935563 [−139.991071, −17.880054]
Constant route 0	1.987948 [0.668060, 3.307836]	−133.216771 [−226.624095, −39.809447]
Constant route 1	1.758035 [0.588921, 2.927149]	−143.299875 [−237.080621, −49.519129]
Load-aware heuristic	0.769377 [0.257290, 1.281465]	−52.075458 [−86.468163, −17.682753]

Source: retained ADR-014 holdout record, E08 (Bhudia, 2026). Each interval uses twelve paired workload-seed means.

The pooled result corresponds to approximately 50.1% higher mean goodput and 75.8% lower mean RTT than nominal-cost OSPF in this scenario mix. However, the direction-specific results explain where the gain arose. When path 0 was impaired, OSPF retained its nominal preferred route while PPO moved away. When path 1 was impaired, both methods used the healthy route 0 and there was no detectable delivery advantage.

Table 5.7: Direction-specific PPO-minus-OSPF results

Impairment condition	Goodput difference, Mbps [95% CI]	RTT difference, ms [95% CI]	Interpretation
Path 0 impaired; six seeds	3.951437 [3.844220, 4.058654]	−157.869625 [−234.711791, −81.027459]	Improvement arose from leaving OSPF's impaired nominal route.
Path 1 impaired; six seeds	0.000141 [−0.000257, 0.000539]	−0.001500 [−0.010225, 0.007225]	Both methods used healthy route 0; no detectable advantage.

The recorded OSPF costs did not track the imposed capacity impairment. The archived configuration derived each cost from 100 times the ceiling of 100 divided by nominal link capacity, then added the link's one-based manifest index as a deterministic perturbation. It permitted one next hop and preferred the distribution-1 path in the unchanged topology. Since OSPF selects routes using its configured link-state costs (Moy, 1998), the experiment establishes an advantage over that specified nominal-cost baseline, not over capacity-aware or reconfigured OSPF. It also does not show that PPO calculates a shortest path faster than OSPF. The improvement concerns delivered traffic and RTT under a particular adverse routing condition.

Packet aggregation provides another important distinction. The recorded PPO foreground totals were 79,769 packets sent and 78,413 received, while OSPF recorded 70,843 sent and 46,503 received. Their packet-weighted losses were approximately 1.699908% and 34.357664%, which differ from the mean-window losses in Table 5.5. Sender duration included control-related time, explaining unequal sent totals despite matched offered schedules. Goodput used actual sender duration and final delivered bytes; the verified drain interval was not added to its denominator.

5.4.3.2 Training Behaviour and Stability

Validation documented progressively stronger preference for the appropriate path. At 128 transitions, correct-route probabilities were approximately 0.615 and 0.590 in the two conditions. At 384 transitions, they reached approximately 0.973 and 0.968. All three examined checkpoints selected the correct route on the measured directional validation cases. These observations support learned directional behaviour, rather than a model that always selected one path.

The validation reward changed much less than the action probabilities. This is plausible once the deterministic route selection is already correct: a stronger preference for the same chosen route need not yield a proportionate increase in the measured reward. Policy probability is therefore evidence about the learned decision distribution, not a calibrated probability that an action is operationally safe.

The six PPO route changes were fewer than the heuristic's 29 changes, but greater than OSPF's zero. The original report identified a control-cost trade-off against OSPF rather than claiming free improvement. The comparison was not a same-policy

safeguard ablation, so it cannot establish that the operational hold/rate layer caused the difference. That causal question remains part of the integrated evaluation planned in Chapter 3.

Subsequent refinement provides useful contrary evidence. The ADR-015 candidate completed 512 fresh transitions and 32 updates across a broader stationary profile set. Its validation reward improved by 0.01313 and its switching count fell from ten to eight across 64 decisions, a 20% reduction. These changes did not meet the declared improvement gates of greater than 0.02 reward gain and at least 25% fewer switches. The incumbent was retained and the new final-test seeds remained closed. This outcome shows that further training produced a modest validation change rather than an automatically qualified replacement (E09).

5.4.3.3 Inference, Control and Observation Timing

The model's numerical computation was short relative to the surrounding measurement and control process. Table 5.8 separates these timings. The first two rows describe recorded fresh-process/warm tests, while the remaining rows describe the live benchmark instrumentation. Their boundaries differ and should not be combined into one latency claim.

Table 5.8: Recorded inference and control timing

Activity	Recorded result	Meaning
Fresh-process wall time	2.24246 s and 2.17448 s	Includes Python startup, imports and verification.
Warm inference, 100 repetitions per process	Mean 0.216 ms and 0.222 ms	Repeated numerical inference on the supplied compatible input.
Live PPO inference	Median 0.956 ms; 95th percentile 1.261 ms	Model decision computation in the recorded benchmark.
PPO control/readback	Median 173.487 ms; 95th percentile 944.242 ms	Action/control verification component.
OSPF control/readback	Median 48.095 ms; 95th percentile 49.593 ms	Baseline control/readback component.

Activity	Recorded result	Meaning
PPO observation/control/ inter-process communication	Median 4.280 s; 95th percentile 5.304 s	Instrumented surrounding loop, not browser display delay.
OSPF observation/control/ inter-process communication	Median 4.099 s; 95th percentile 4.528 s	Corresponding baseline loop.

These results explain why a sub-millisecond warm inference does not justify describing the complete orchestrator as a sub-millisecond controller. They also do not establish a device-to-dashboard latency distribution. The telemetry integration records establish identity/value consistency and bounded drain, but a separately timestamped reading-to-render experiment would be required to quantify browser-display delay.

5.4.3.4 Application and Recovery Evaluation

The consolidated module campaign demonstrated the operation of telemetry, manual controls, measured alerts, reports and timed recovery in the stated laboratory. Its alert test recorded a 12.934441-second interval from workload stop to the resolving observation. This interval included the sustained recovery rule; it is not a pure event-publication delay or a claim that all congestion incidents resolve within that time.

The six override cases restored owned flows to zero, with verification times between approximately six and nine seconds in the final retest. The results support asynchronous recovery for those scenarios. They do not establish instantaneous undo, recovery from every filesystem crash point or a universal recovery-time objective after host power failure.

The default consolidated software record reported 2,028 backend tests passing with 89 opt-in skips, 487 frontend tests and 41 browser tests. An additional alert-focused record reports 2,054 backend tests with 58 opt-in skips at its own review point. These totals describe different source/test states and must not be added together or interpreted as counts of unique experimental repetitions. The dated consolidated campaign remains the primary system-level summary used here.

5.4.4 Results Presentation and Interpretation

The results support three conclusions at different levels. First, the implemented operator pipeline can preserve resource identity and measurement meaning through the tested collection, storage and display paths. Second, the bounded manual-control workflows can execute and recover supported actions with retained evidence. Third, the selected PPO checkpoint improved delivery over nominal-cost OSPF in the recorded stationary impairment experiment.

Those conclusions do not collapse into a claim of complete proactive autonomy. The matched routing benchmark and the operator testbed use different control interfaces; compatible live observation and calibrated dispatch remain additional requirements. The consolidated acceptance record also leaves the larger-topology and complete low/ramp/burst/overload matrix unperformed as a new measured campaign. A modelled counterpart or a historical hash check does not fill that gap.

The evidence is therefore presented with its sample unit and measurement boundary. Mean-window loss is not pooled packet loss; ICMP RTT is not one-way application latency; an inference timing is not reading-to-display delay; and a fixture-backed interface test is not a physical-network experiment. These distinctions make the results useful for reproduction and comparison rather than merely favourable in appearance.

5.5 Version Control and Implementation Evidence

The configured Project repository contains the source, dependency locks, migrations, experiment procedures, architecture decisions and engineering records. The documentation review used commit 412cb85882c4838546837688ad3800177da778bc as the source snapshot. The active local branch was main, and no local release tags were present. An unauthenticated web request returned 404 during preparation, so access by the supervisor must be confirmed through the repository's actual permissions rather than presumed from the presence of a URL.

The repository history records successive implementation changes, including the September 12 module/acceptance increment and September 17 model/documentation updates. Several commit messages are terse, so the architecture decisions, module runbooks and validation records provide much of the explanatory traceability. The

available history does not establish a branch-per-feature or pull-request review process, and such a process is not claimed in this chapter.

Dependency locks, environment definitions and source/image hashes support version identification. Database migrations describe schema evolution, while runbooks identify the commands and prerequisites used by particular campaigns. The raw model archive is deliberately separate from ordinary source history. The local checkpoint, plan, selection and five raw trace hashes matched their recorded identifiers during review. Their retention and authorized transfer remain necessary for another reviewer to reproduce the analysis independently.

Table 5.9 identifies the principal evidence records used in this chapter. These are student-project records cited collectively as Bhudia (2026); they are not independent peer-reviewed literature. A recorded test outcome is attributed to its source, and any future rerun should retain a new dated evidence package rather than overwrite the original result.

Table 5.9: Implementation and evaluation evidence register

ID	Repository record	Evidence used
E01	docs/project/FoundationSecurity-Step1-Step2.md	Session/membership design, fixture scope and foundation regression.
E02	docs/project/Emulation-Backend-Live-Validation.md	Resource binding, source/storage/WebSocket checks, partial replay and measurement corrections.
E03	docs/project/ManualExecution-Step5-Step6.md	Manual controls, concurrency, worker recovery and measured workload effects.
E04	docs/project/SimulationOperator-Step11-Step12.md	Deterministic simulation, accounting and configured-evidence integration.

ID	Repository record	Evidence used
E05	docs/project/OperatorCompletion-ADR018.md	Selected-probe paths, confined inference and operator/override workflows.
E06	docs/project/ModulesAcceptance-Step13-Step14.md; backend/scripts/acceptance/VALIDATION.md	Consolidated pass/fail/blocked matrix, raw reported values, defects and focused retests.
E07	ai-engine/ADR014-RESULTS-001.md	Training transitions, validation progression and original budget/selection shortfall.
E08	ai-engine/ADR014-HOLDOUT-RESULTS.md	Held-out methods, sample counts, paired intervals, scenario analysis and timings.
E09	docs/project/ModelRefinement-ADR015-ADR016.md	Later refinement outcomes and incumbent-preservation limits.
E10	docs/project/SafetyAutonomy-Step9-Step10.md	Conditional safety design and open autonomous-activation gates.
E11	docs/project/Frontend-Step13-ADR019.md	Interface, download-integrity, browser and bundle checks.
E12	deploy/README.md; deploy/OPERATIONS.md; docs/project/DevelopmentJournal.md	Build/deployment definitions, operations checks and unclosed recovery acceptance.

Source provenance supports accountability but does not replace access to the underlying files. For final reproducibility, the authorized reviewer needs the matching checkpoint, frozen source/image references, workload plans, raw measurements and analysis scripts, in addition to access to the repository. Secrets, live credentials and unrelated institutional data are not part of that handover.

5.6 Evaluation Results, Discussion and Limitations

5.6.1 Objective i: Congestion Indicators and Traffic-Management Challenges

The implementation established a measurement pipeline for utilization, queue occupancy, RTT and packet-loss observations, supported by separate endpoint delivery records in the routing experiment. Independent counter and queue checks addressed the risk of mistaking convenient output fields for valid measurements. The corrected partial-publication test also showed that preserving identity across replay was necessary for a trustworthy historical dataset.

The evidence characterizes the bounded laboratory and the specific workloads that were actually run. It does not complete the proposed characterization of every normal, burst and sustained-load family. The consolidated campaign explicitly retained blocked cases for the broader measured workload matrix and larger topology. Objective i is therefore supported by substantial measurement implementation and case evidence, but its complete proposed workload coverage remains partial.

5.6.2 Objective ii: Existing Intelligent Network-Management Solutions

Chapter 2 compared three commercial platforms and four research implementations, establishing the distinctions between assurance, performance prediction, demand allocation and routing control. Those distinctions informed the implementation: the interface links observations to action evidence, the simulator labels predictions separately, and the routing evaluation includes a load-aware non-learning comparator.

The contribution is a documented comparative analysis rather than a claim that NANFO outperforms the commercial systems. The product accounts do not expose an equivalent laboratory protocol, and the research studies evaluate different action spaces and outcomes. ENERO's hybrid search approach supports the need for competent non-learning alternatives, while RouteNet-Fermi's variable prediction error motivates caution when model output is used for control (Almasan, Xiao, et al., 2022; Ferriol-Galmés et al., 2023).

5.6.3 Objective iii: Limitations in Timing, Stability and Reproducibility

The project identified and retained several limitations that directly affect interpretation. The collector's pending batch is memory-only; packet-level and

modelled evidence have different meanings; checkpoint qualification does not automatically install a live control provider; and one trained initialization cannot establish robustness to training randomness. These findings refine the gap identified in the literature into concrete integration and evaluation requirements.

The recorded benchmark shows a useful routing trade-off, with improved delivery but more route changes than a fixed nominal OSPF route. It does not measure proactive lead time. The timing records also show that model computation is only a small part of the surrounding control and observation loop. These results justify evaluating complete intervention timing rather than using neural inference speed as a substitute (Agarwal et al., 2021; Farrel, 2024).

5.6.4 Objective iv: Design and Development of the Integrated Prototype

NANFO implemented the principal operator functions: authenticated scoped access, inventory, measured telemetry, numerical simulation, supported manual actions, selected-probe capture, diagnostic inference, timed restoration, alerts, audit history and reports. Their implementation is supported by component, integration and laboratory records rather than interface appearance alone.

The integration remains incomplete at the safeguarded autonomous-control boundary. The PPO benchmark demonstrated a useful learned policy in the matched experiment, while the operator application demonstrated guarded manual workflows and historical inference. The available evidence does not establish that those components have been validated together as a calibrated live autonomous controller. Objective iv is therefore partially achieved at the complete research-system level, despite substantial verified functionality in its constituent components.

5.6.5 Objective v: Comparative Performance Validation

The selected checkpoint passed its stated held-out goodput and RTT criterion against nominal-cost OSPF. The directional results explain the mechanism: improvement occurred when OSPF retained the impaired path and disappeared when both methods used the healthy path. This is a defensible, bounded result rather than a universal ranking of DRL and conventional routing.

The result is consistent with the broader motivation for state-dependent traffic engineering, but it does not reproduce the topology-generalization studies reviewed in

Chapter 2. DRL–GNN evaluated optical allocation over many unseen topologies, and GTSR evaluated simulated packet routing with different model and workload assumptions (Almasan, Suárez-Varela, et al., 2022; Li et al., 2025). NANFO's contribution is its own measured two-path comparison and the accompanying operator evidence, not the transfer of those studies' broader claims to a small campus emulator. Complete fulfilment of Objective v still requires the newly planned workload-family extension, additional trained initializations, the same-policy safeguard ablation and intervention-timing assessment. These are outstanding promised evaluations, not optional future enhancements. Their absence is recorded separately from the successful stationary benchmark.

5.6.6 Validity and Generalization Limits

Internal validity is constrained by one shuffled method-session order, shared-host timing and the overhead of measurement/control. Constant route 0 and OSPF used the same preferred path and had similar delivery/loss behaviour but different average RTT, indicating session or instrumentation effects. Shuffling once did not eliminate time drift or produce a counterbalanced replication.

Statistical interpretation is constrained by twelve paired workload means and one trained initialization. The reported Student-t intervals assume an adequate model for the seed-level differences, while the two impairment directions form a heterogeneous mixture. Multiple reported 95% intervals are not simultaneous 95% coverage of every comparison. The findings should therefore be read with their direction-specific results and experimental scope rather than as a single general improvement percentage.

Construct validity depends on the definitions used. ICMP RTT represents probe behaviour, not every application's latency. Verified-drain loss improves packet accounting but does not provide unavailable late-packet counts. Route-change counts depend on the route-0 episode reset. The finite-buffer fluid model does not reproduce all TCP, packet scheduling or controller-convergence effects. No independent physical calibration error or measured device-to-browser delay distribution was established by the retained summaries.

External validity is limited by the five-node/four-host topology, synthetic workloads, legacy isolated controller dependencies and absence of a live institutional

deployment. Physical energy savings, wireless communication range and sensor-circuit performance are not relevant measured outputs of this prototype and are not inferred. Participant-based usability and complete host-power-loss recovery also remain unverified.

Finally, reproducibility depends on access to the raw archive and repository. The local read-only check reproduced delivery measures and paired intervals from the archived endpoint and probe evidence, but it did not execute the model or recreate the physical laboratory. Other historical campaigns are represented here by retained reports, and some temporary raw-output paths are no longer available. Preserving the accessible experiment artefacts and arranging authorized reviewer access are therefore necessary parts of the evidence handover. Within these boundaries, the implementation and recorded results establish a useful measured routing prototype and a substantial operator workflow, while leaving the complete proactive autonomous claim open to further validation.

References

- African Union Commission. (2022). *Digital education strategy and implementation plan 2023–2028*. https://au.int/sites/default/files/documents/42416-doc-1_DES_EN_-_2022_09_14.pdf
- Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A. C., & Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 29304–29320. <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- Almasan, P., Suárez-Varela, J., Rusek, K., Barlet-Ros, P., & Cabellos-Aparicio, A. (2022). Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196, 184–194. <https://doi.org/10.1016/j.comcom.2022.09.029>
- Almasan, P., Xiao, S., Cheng, X., Shi, X., Barlet-Ros, P., & Cabellos-Aparicio, A. (2022). ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning. *Computer Networks*, 214, Article 109166. <https://doi.org/10.1016/j.comnet.2022.109166>
- Arista Networks. (2026). *CloudVision data sheet*. https://www.arista.com/assets/data/pdf/Datasheets/EOSCloudVision_DataSheet.pdf
- Bhudia, D. H. (2026). *NANFO: Source code and engineering records* (Commit 412cb85882c4838546837688ad3800177da778bc) [Computer software and technical reports]. GitHub. <https://github.com/dhb001/NANFO>
- Brunke, L., Greeff, M., Hall, A. W., Yuan, Z., Zhou, S., Panerati, J., & Schoellig, A. P. (2022). Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5, 411–444. <https://doi.org/10.1146/annurev-control-042920-020211>

Cisco Systems. (2025). *Cisco Catalyst Assurance user guide, release 2.3.7.x: Monitor and troubleshoot network health*.

https://www.cisco.com/c/en/us/td/docs/cloud-systems-management/network-automation-and-management/catalyst-center-assurance/2-3-7/b_cisco_catalyst_assurance_2_3_7 Ug/b_cisco_catalyst_assurance_2_3_6 Ug_chapter_0110.html

Cisco Systems. (2026). *Cisco Catalyst Center 2.3.7 data sheet*.

<https://www.cisco.com/c/en/us/products/collateral/cloud-systems-management/dna-center/nb-06-dna-center-data-sheet-cte-en.html>

Coleman, J., Vacanti, D., Johnson, C., Singh, P., Wester, J., Neverdal, C., Firlit, M., Gilb, T., & Tendon, S. (2025). *The Kanban guide*.

<https://kanbanguides.org/the-kanban-guide>

Farrel, A. (Ed.). (2024). *Overview and principles of Internet traffic engineering* (RFC 9522). Internet Engineering Task Force. <https://doi.org/10.17487/RFC9522>

Ferriol-Galmés, M., Paillisse, J., Suárez-Varela, J., Rusek, K., Xiao, S., Shi, X., Cheng, X., Barlet-Ros, P., & Cabellos-Aparicio, A. (2023). RouteNet-Fermi: Network modeling with graph neural networks. *IEEE/ACM Transactions on Networking*, 31(6), 3080–3095. <https://doi.org/10.1109/TNET.2023.3269983>

International Organization for Standardization, & International Electrotechnical Commission. (2023). *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model* (ISO/IEC Standard No. 25010:2023).

<https://www.iso.org/standard/78176.html>

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering—Life cycle processes—Requirements engineering* (ISO/IEC/IEEE Standard No. 29148:2018).

<https://www.iso.org/standard/72089.html>

- International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software and systems engineering—Software testing—Part 1: General concepts* (ISO/IEC/IEEE Standard No. 29119-1:2022).
<https://www.iso.org/standard/81291.html>
- Juniper Networks. (2025). *Marvis AI assistant datasheet*.
<https://www.juniper.net/us/en/products/cloud-services/marvis-ai-assistant-datasheet.html>
- Li, X., Li, J., Zhou, J., & Liu, J. (2025). Towards robust routing: Enabling long-range perception with the power of graph transformers and deep reinforcement learning in software-defined networks. *Electronics*, 14(3), Article 476.
<https://doi.org/10.3390/electronics14030476>
- Moy, J. (1998). *OSPF version 2* (RFC 2328). Internet Engineering Task Force.
<https://doi.org/10.17487/RFC2328>
- Object Management Group. (2017). *Unified Modeling Language (UML), version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1/PDF>
- OWASP Foundation. (2020). *Web security testing guide* (Version 4.2).
<https://owasp.org/www-project-web-security-testing-guide/v42>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv.
<https://doi.org/10.48550/arXiv.1707.06347>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*.
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>

Appendix 1: Gantt Chart

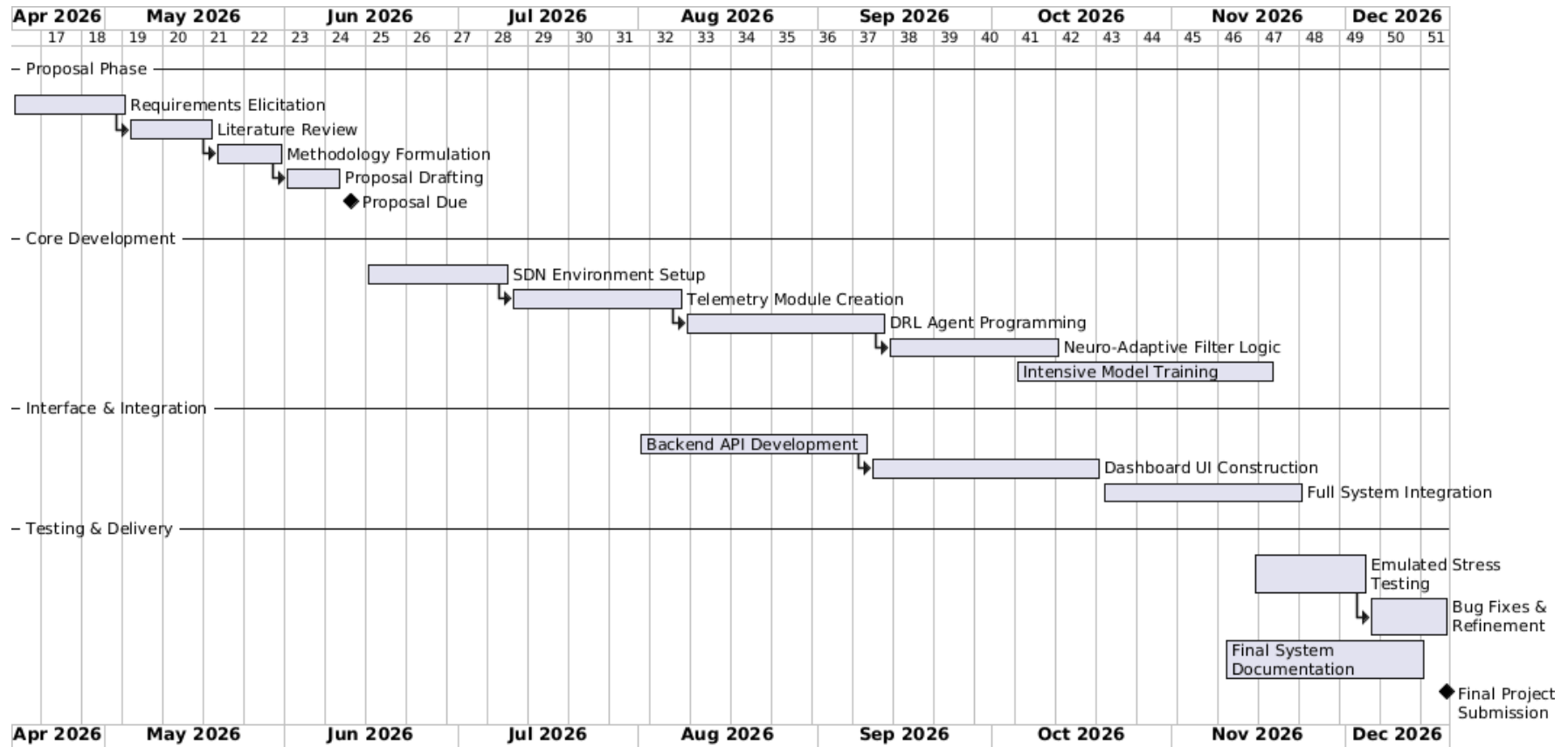


Figure A1. Original project time schedule